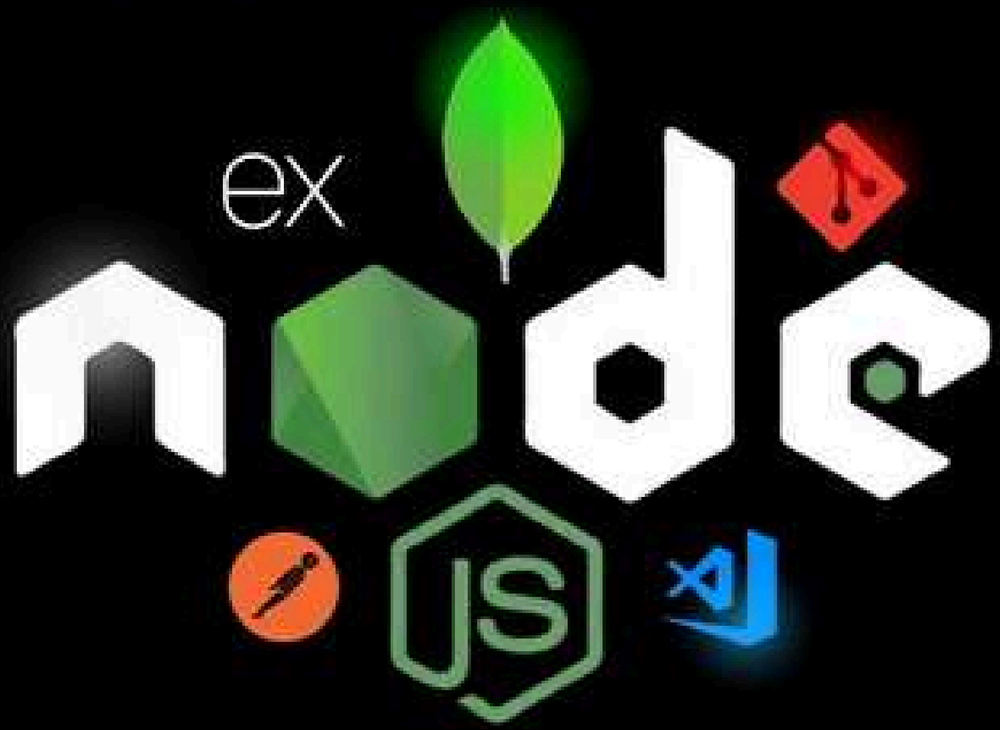
AKATSUKI CODING CLUB

PRESENTS

NODEVEMBER 2.0

THE AWAKENING OF NODE.JS



Build, test, deploy and certify – Nodevember 2.0 is your gateway to becoming a backend pro!

NODEVEMBER 2.0

The Awakening of Node.Js

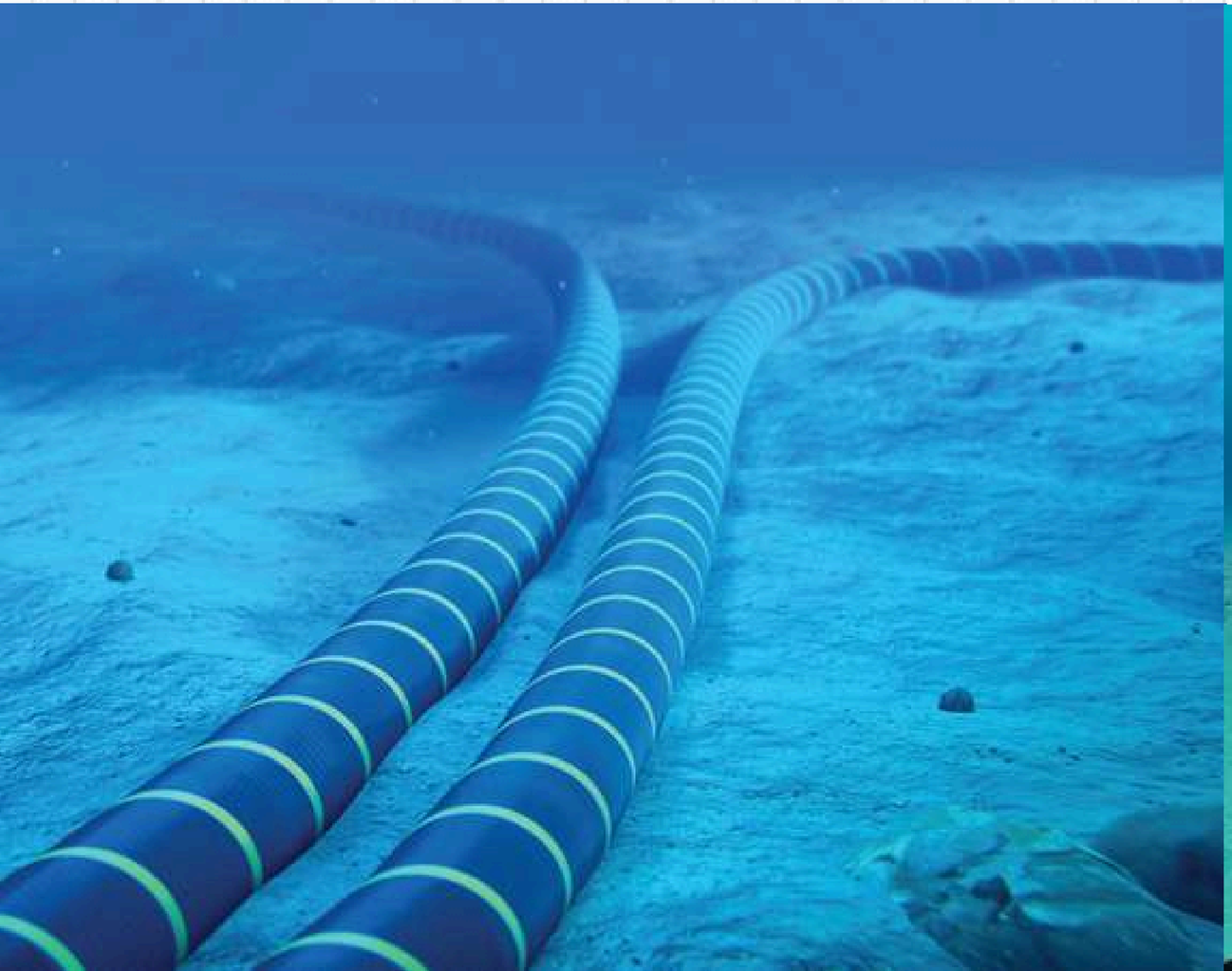

Server is Running on PORT 3000

BEFORE WE START...

HOW INTERNET WORKS ?

“Internet is not in the Clouds, its under the Ocean”

www.submarinecablemap.com



WHAT IS IP ADDRESS ?

1. Numerical 32-bit address assigned to each device
2. Ex. 192.168.1.1
3. IP address decides which device to send data

Local IP

Local IP

Global IP

Router

Local IP

Local IP

IP
123.123.1.2

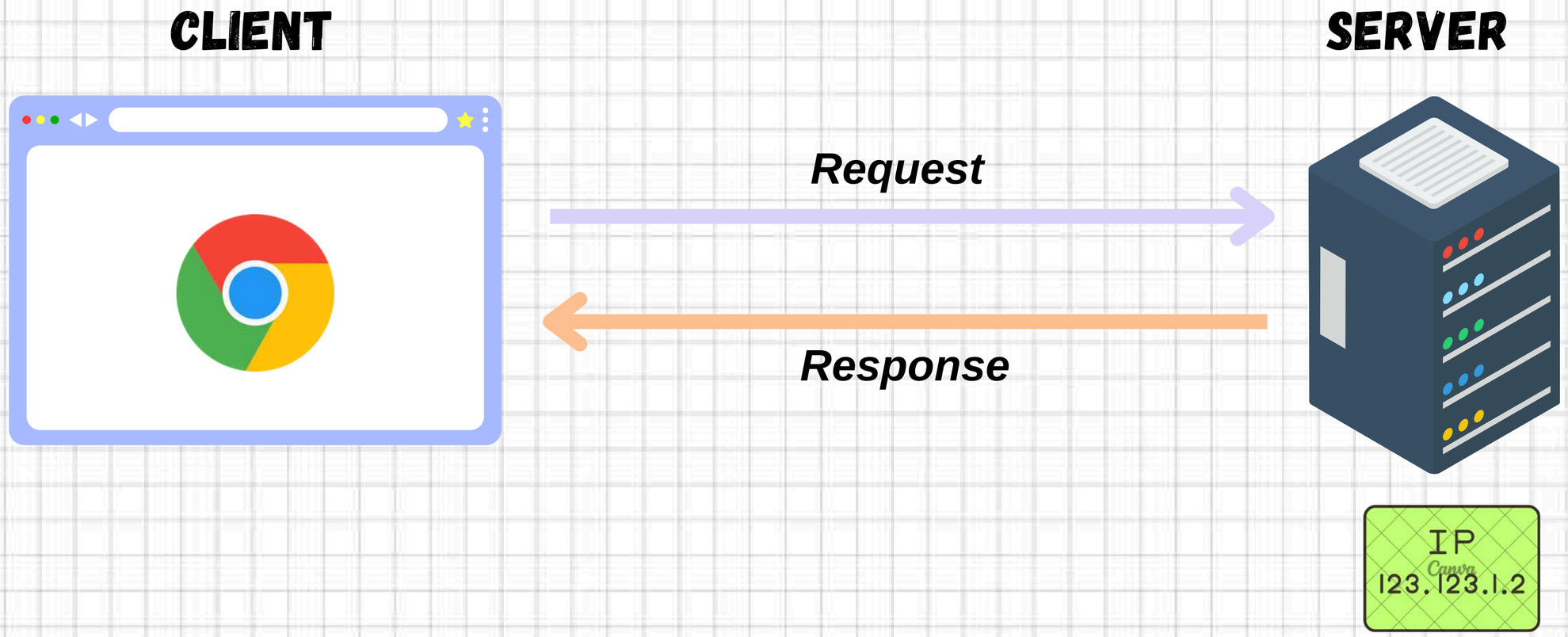
Server

Internet

WHAT IS PORT NUMBER?

1. Decides which application to send data
2. Port number is 16-bit number
3. Total Port Numbers = 2^16 = 65535
4. 0 - 1023 are Reserved Port (Ex. HTTP:80, SMTP: 25)
5. 1024 - 49151 are Application Port
   Ex: MongoDB: 27017, SQL: 1433

CLIENT-SERVER ARCHITECTURE



HOW CLIENT KNOWS IP ADDRESS OF SERVER ?

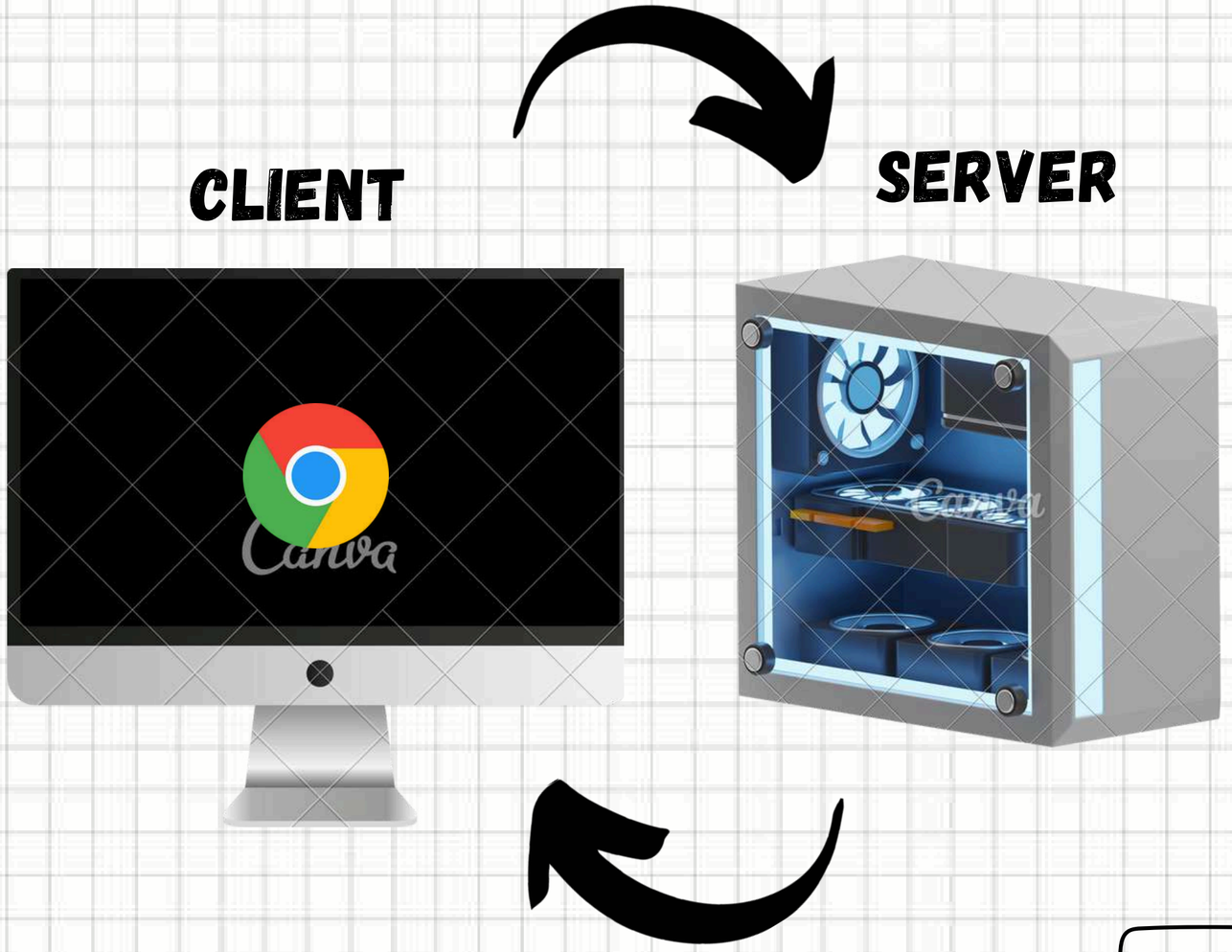
Domain Name Server (DNS)

DNS

google.com	→	142.250.192.14
instagram.com	→	163.70.140.174
x.com	→	104.244.42.65
youtube.com	→	142.251.42.110
rcpit.ac.in	→	216.10.243.24

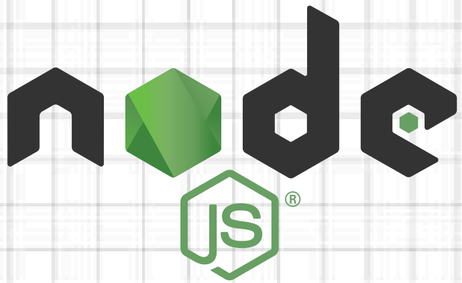
WHAT IS LOCALHOST ?

1. Loopback Address (127.0.0.1)
2. Local Testing and Development
3. No Network Required
4. Port Usage



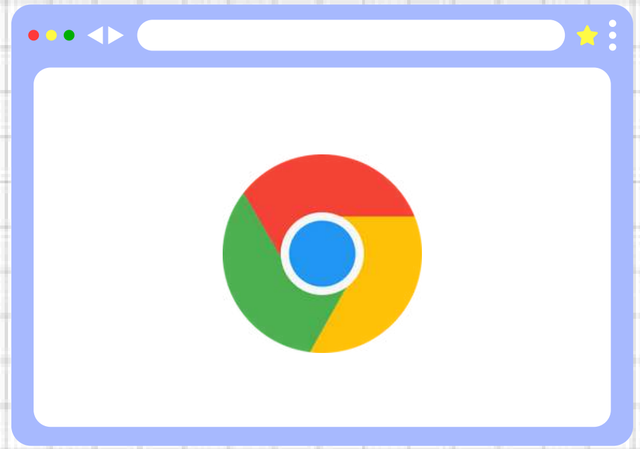
LETS START...

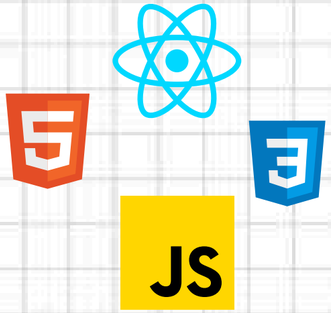
WHAT IS NODE.JS ?



BEFORE WE START

CLIENT

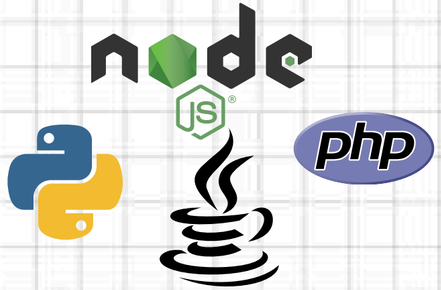




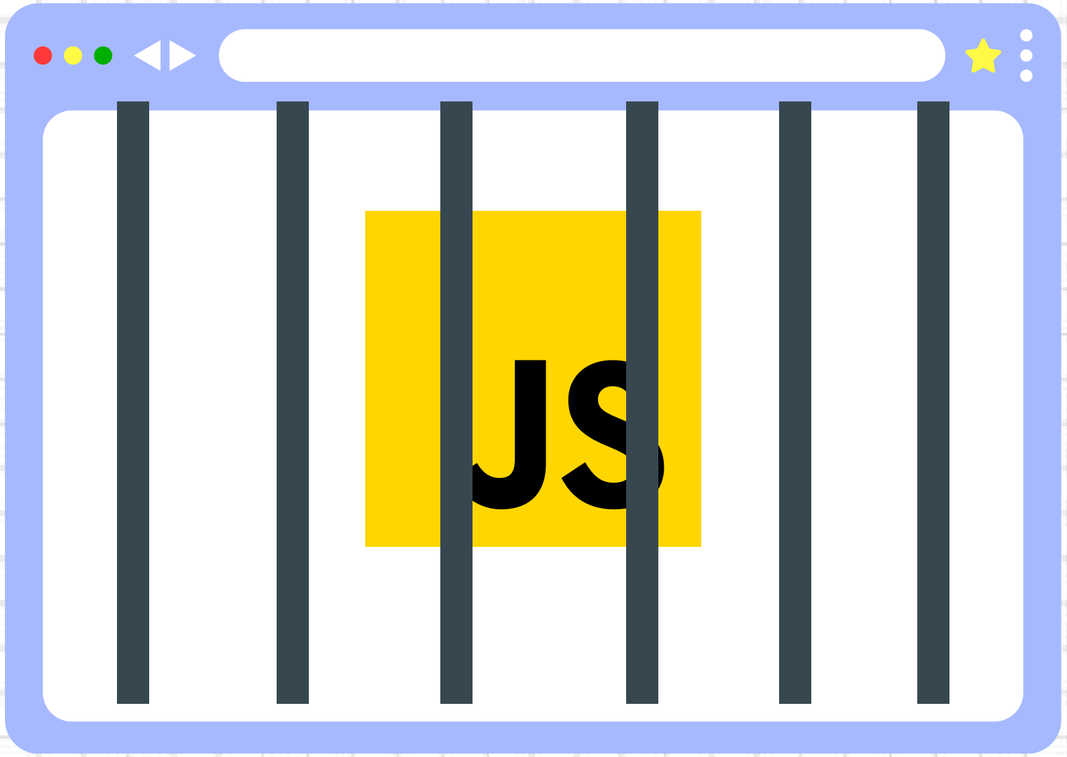
Request

Response

SERVER

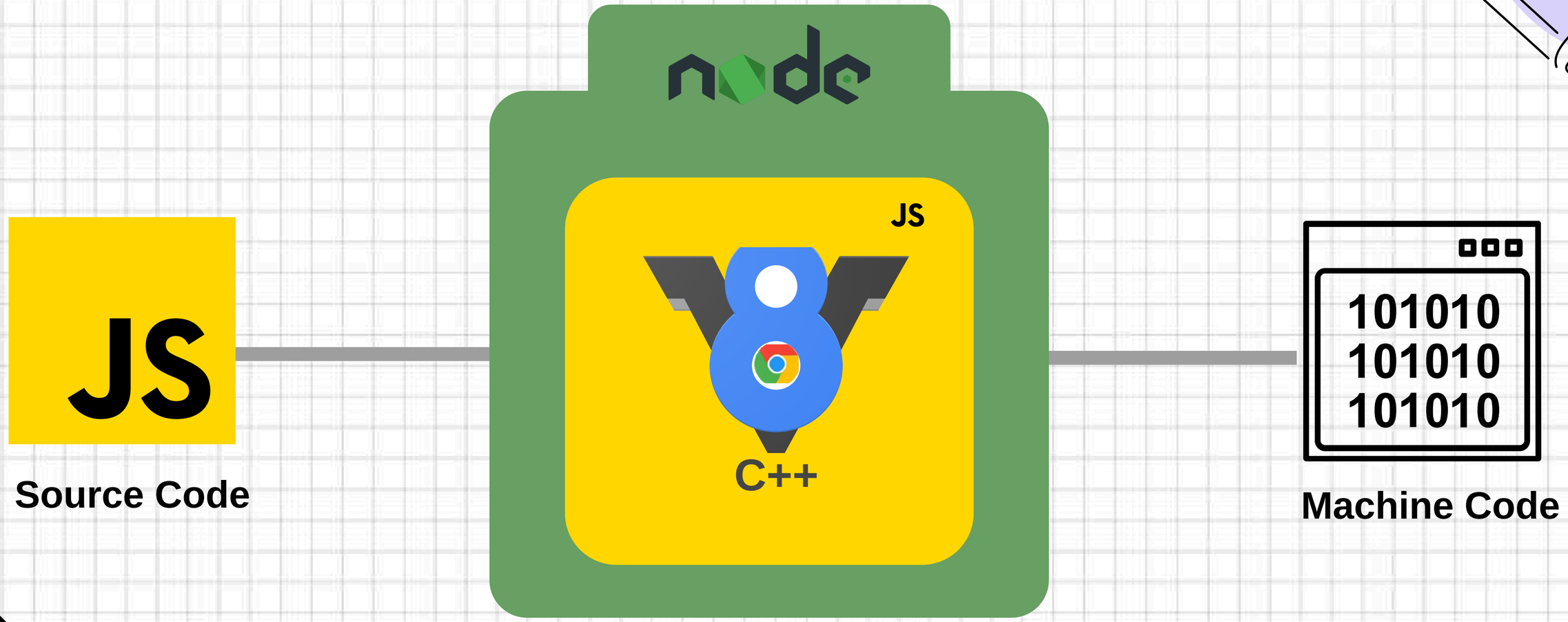


JAVASCRIPT IN BROWSER



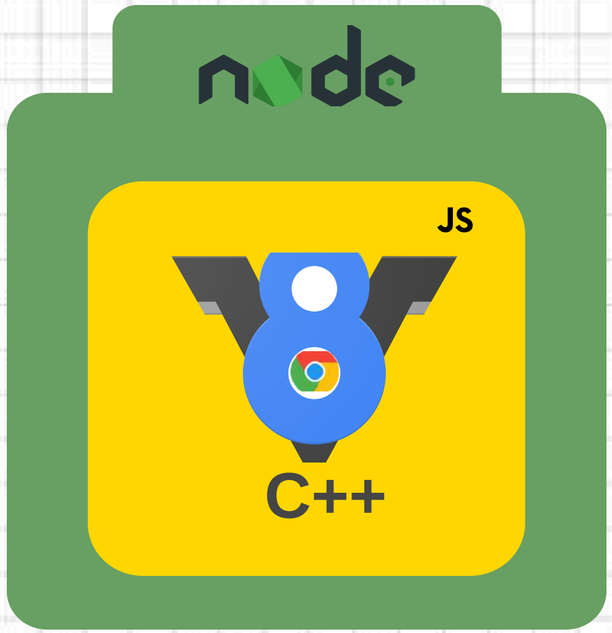
V8 Engine compiles JavaScript code into machine code

ENTER NODE.JS



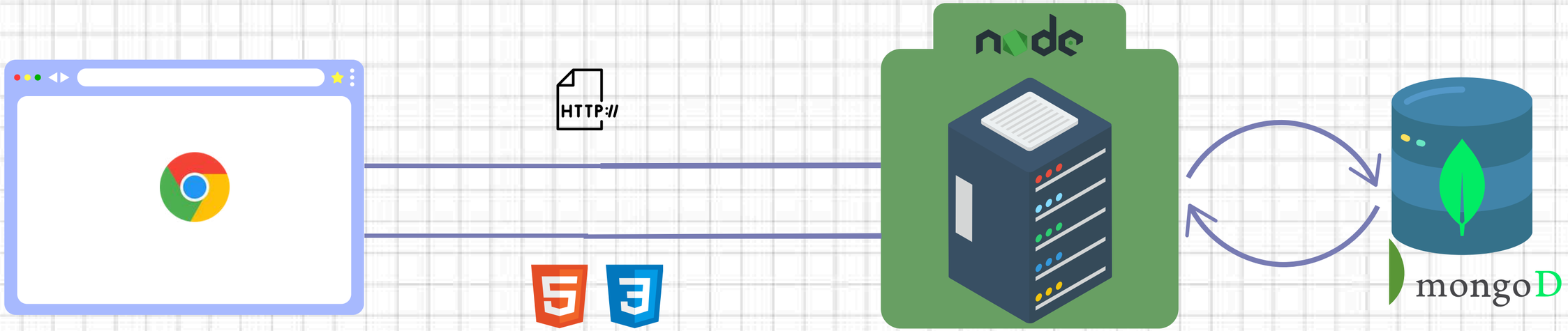
Node.js is written in C++

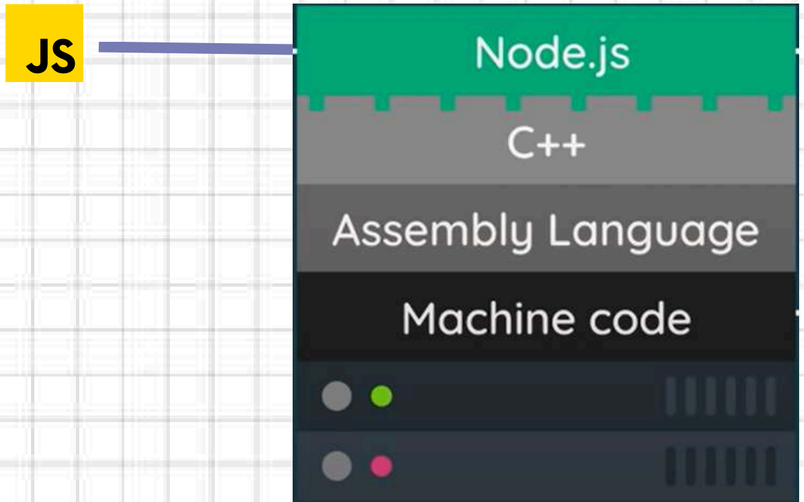
NODE IS MORE THAN V8...



- Adds more features to JavaScript
- Read & write files on a computer
- Connect to a database
- Act as a server for content

THE ROLE OF NODE.JS





APPLICATION OF NODE.JS?

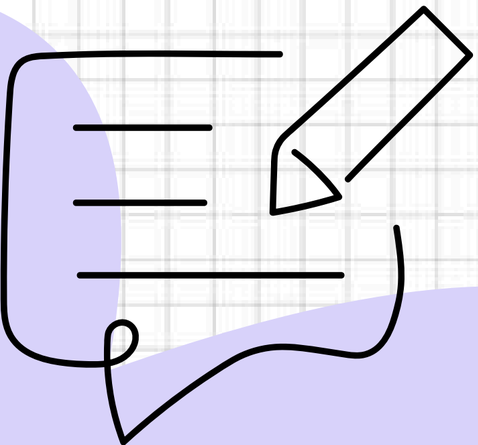
Uber

USE BY

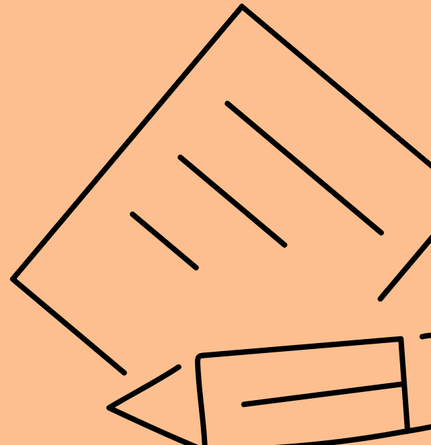
ebay

WHAT YOU WILL LEARN...

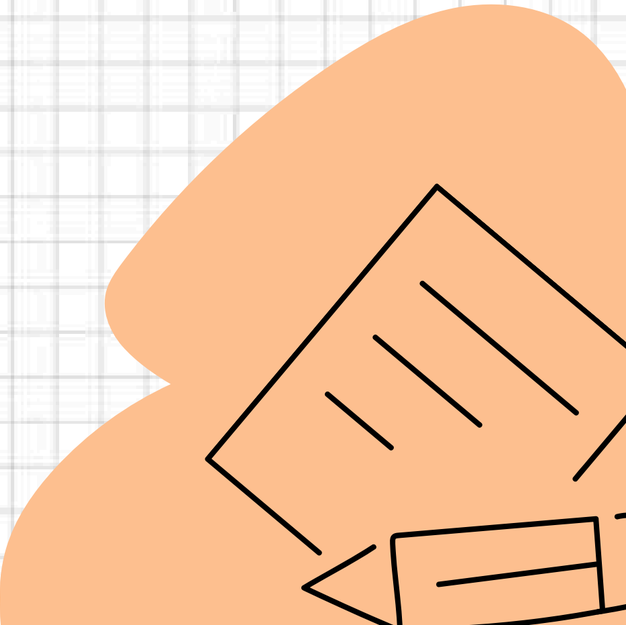
1. Install Node.js & use NPM
2. Learn Express.js framework
3. How to create a server using Express.js
4. How to use postman for API testing
5. How to use MongoDB (NoSQL database)
6. Put everything together to make a Backend Project

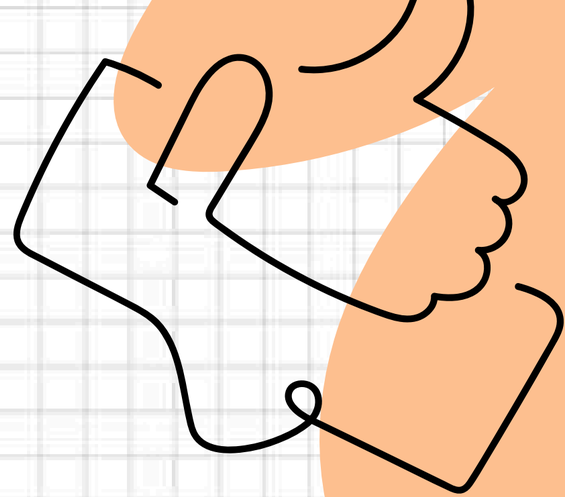


INSTALLING NODE.JS



WHAT IS NPM?



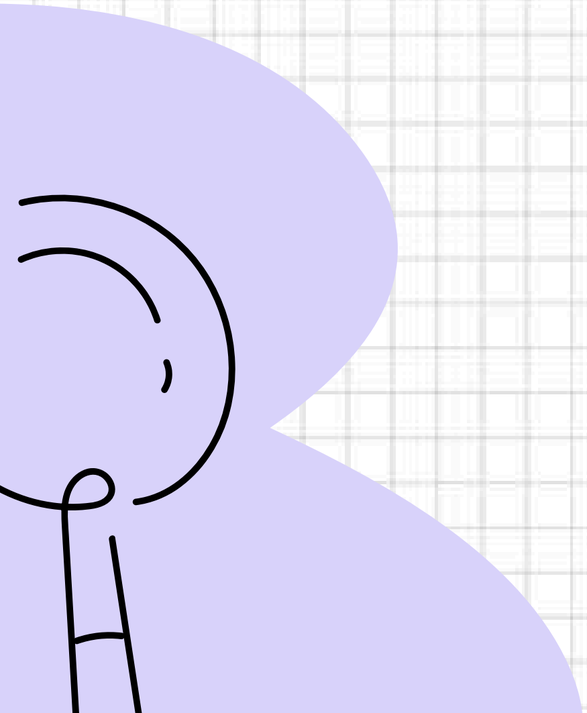


BEFORE WE START...

LET'S LEARN JS FUNDAMENTALS...

JS

WHAT IS JS?



THE AKATSUKI CODING CLUB

JAVASCRIPT FUNDAMENTALS

01

- Variables
- Loops
- Conditionals

02

- Functions
- Arrow func
- Arrays
- Objects

03

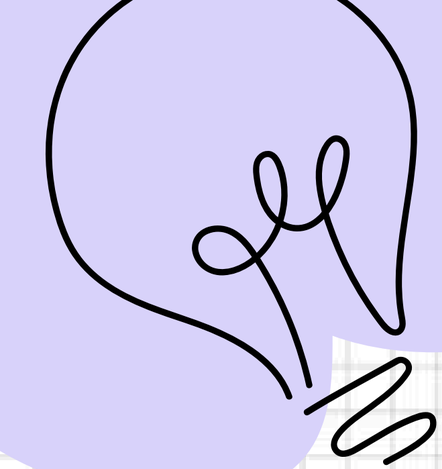
- Callbacks
- Promises
- Async/await

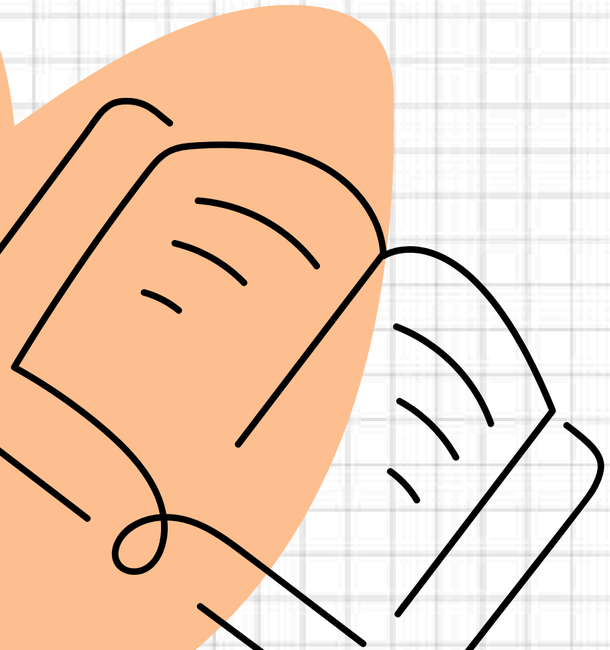
04

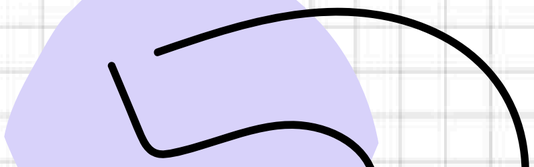
- try-catch
- Import-Export

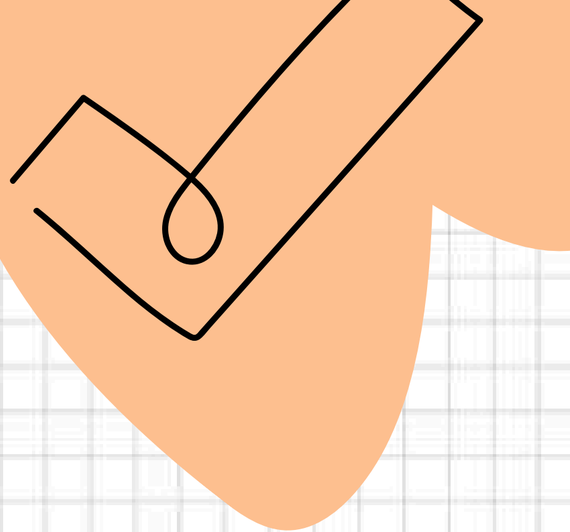
THE AKATSUKI CODING CLUB

VARIABLES, LOOPS AND CONDITIONALS









VARIABLES IN JS

VAR

variables can be re-declared and updated.A global scope varibale

LET

variables cannot be re-declared but can be updated. A block scope varibale

CONST

variables cannot be re-declared or updated. A block scope variable

TYPES OF LOOPS

FOR LOOP

The for loop, for example, is handy for **iterating through arrays** or executing code a **specific number of times**.
It has three parts: **initialization**, **condition**, and **iteration.**

WHILE LOOP

The while loop, is useful for executing code while a **specified condition** is true.

The **while** loop, runs while a **specified condition** is true.

DO WHILE LOOP

The do-while loop guarantees that the loop's code block **executes at least once** before the condition is checked.

FOR & WHILE LOOP EXAMPLE

For Loop

```
//for loop
for(let i=0;i<10;i++){
    console.log(i);
}
```

While Loop

```
//while loop
let n = 12;
let i = 10;

while (i < n) {
    console.log(i);
    i++;
}
```

CONDTIONALS

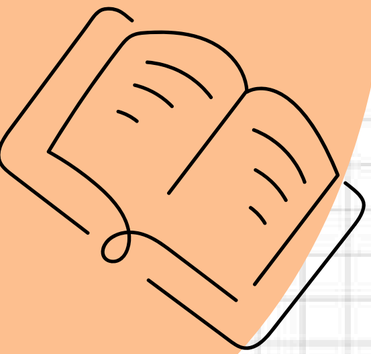
Conditional statements like if and else help us control the flow of our programs based on certain conditions. They're crucial for executing different blocks of code based on whether conditions are met."

Syntax:

```
if (condition) {
  // Code to be executed if the condition is true
  }
else {
   // Code to be executed if the condition is false
  }
```

THE AKATSUKI CODING CLUB

FUNCTIONS

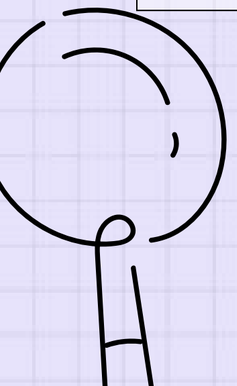


WHAT AND WHY ?

Functions in J**avaScript** are self-contained **blocks of code** designed to **perform a specific task** or set of tasks

Functions allow you to **encapsulate** a piece of code, give it a name, and **reuse it throughout** your program.

```
function functionName(parameter1, parameter2) {
    // Function body - code to be executed
    // It can use the parameters to perform operations
```



ARROW FUNCTIONS

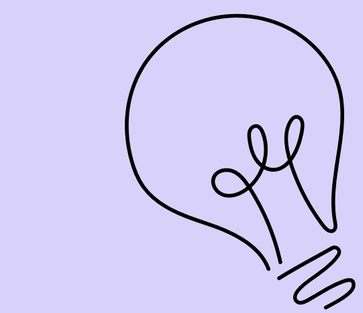
Concise Syntax: Arrow functions have a **shorter syntax** compared to regular functions, especially for single-expression functions.

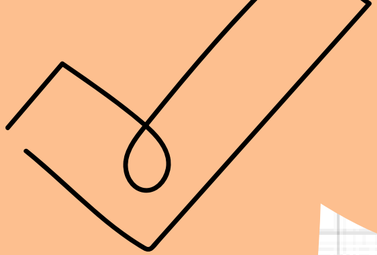
Arrow (**=>**): Signifies the **creation** of an arrow function.

```
const functionName = (parameter1, parameter2) => {
  // Function body - code to be executed
  // It can use the parameters to perform operations

  return result; // returns a value
};
```

THE AKATSUKI CODING CLUB

ARRAYS





WHAT IS AN ARRAY?

JavaScript arrays are versatile **data structures** with various functions and iteration methods that allow manipulation and traversal of array elements.

Arrays are **ordered collections** of values stored under a **single variable** name.

- Arrays in JavaScript are created using square **brackets** [ ].

Examples:

```
//Array
const friends=['Bhavesh','chandrakant','Bhumika']//Array of Strings
const numbers=[1,2,3,4,5]; //Array of numbers
const mixed =['Bhavesh',141,true]; //Array of mixed data types
const emptyArray=[]; //Empty array declaration
```

ITERATION METHOD

forEach():

```
//1) forEach
const anime=['Naruto','DemonSlayer','DeathNote'];
anime.forEach(animeS => { console.log(anime);
});
```

map():

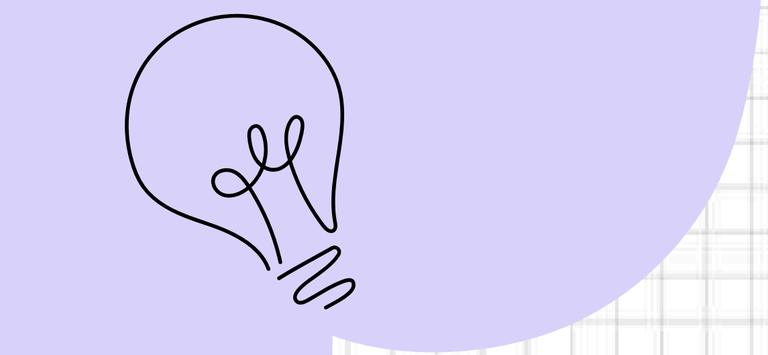
```
//2) Map
const chars =anime.map(ch => ch + " anime");
console.log(chars);
```

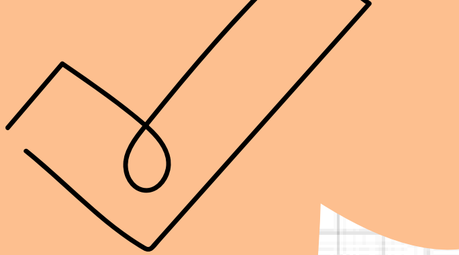
filter():

```
//3)filter
let number= [1, 2, 3, 4];
let evens = number.filter((num) => num % 2 === 0);
console.log(evens);
```

SOME ARRAY METHODS

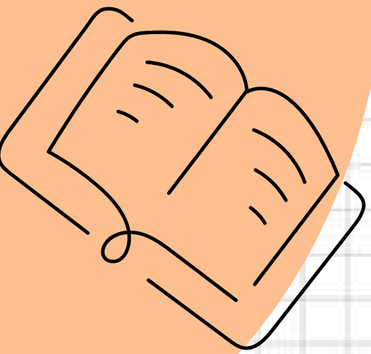
```
var num = [1, 2, 3, 4];
num.at(1)                 // 2
num.push(5)               // Add element to the end: [1, 2, 3, 4, 5]
num.pop()                 // remove last element: [1, 2, 3]
num.fill(1)               // fill every element: [1, 1, 1, 1]
num.shift()               // remove first element: [2, 3, 4]
num.unshift(5)            // Add element to beginning: [5, 1, 2, 3, 4 ]
num.reverse()             // sort in descending order: [4, 3, 2, 1]
num.includes(2)           // is array contains a specified value: true
num.map( item => 2*item)              // Map elementsz; [2, 4, 6, 8]
num.filter( item => item > 2)         // filter elemet: [3, 4]
num.find(item => item > 2)            // Find element : 3(first match)
num.every(item => item > 0)           // true
num.findIndex(item => item === 2)     // 1
num.reduce( (prev, curr) => prev + curr, 0) // 10
num.toString();                 // convert to string
num.join(" * ");                // join: "1 * 2 * 3 * 4"
num.splice(2, 0, "i", "p"); // add elements : [1, 2, 'i', 'p', 3, 4]
num.slice(1,4);                 // slice elements from [1] to [4-1]
num.sort();                     // sort string alphabetically
x.sort(function(a, b){return a - b});    // numeric sort
x.sort(function(a, b){return b - a});    // numeric descending sort
x.sort(function(a, b){return 0.5 - Math.random()}); // random sort
```





THE AKATSUKI CODING CLUB

OBJECTS



FEATURES

1. **Key-Value Pairs**

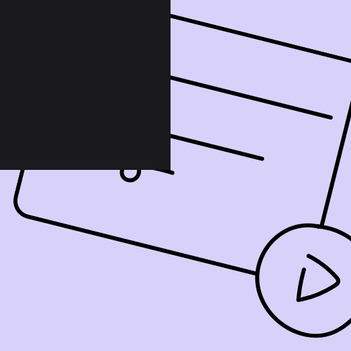
2. **Nested Objects**

3. **Dynamic Properties:**
   Properties can be added, modified, or deleted from objects dynamically.

4. **Accessing Properties:**
   - Dot notation: object.property
   - Bracket notation: object['property']

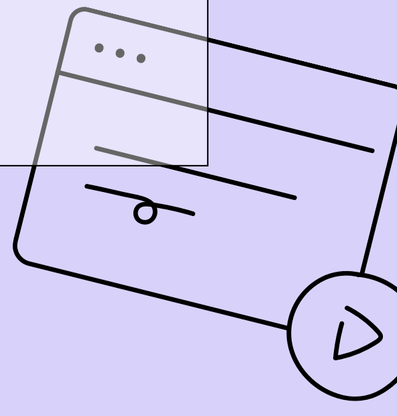
SYNTAX:

```
const obj = {
    key1: value1,
    key2: value2,
    key3: function() {
        // function code here
    },
    nestedObj: {
        nestedKey1: nestedValue1,
        nestedKey2: nestedValue2,
        nestedKey3: function() {
            // function code here
        }
    }
};
```

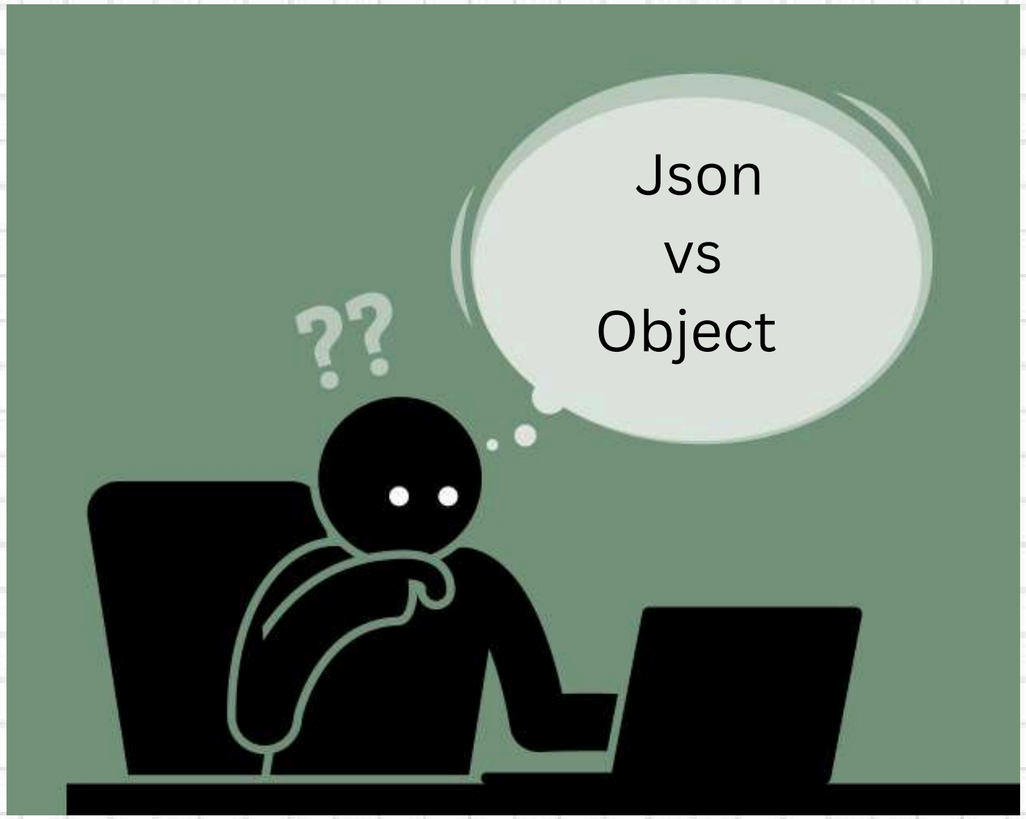


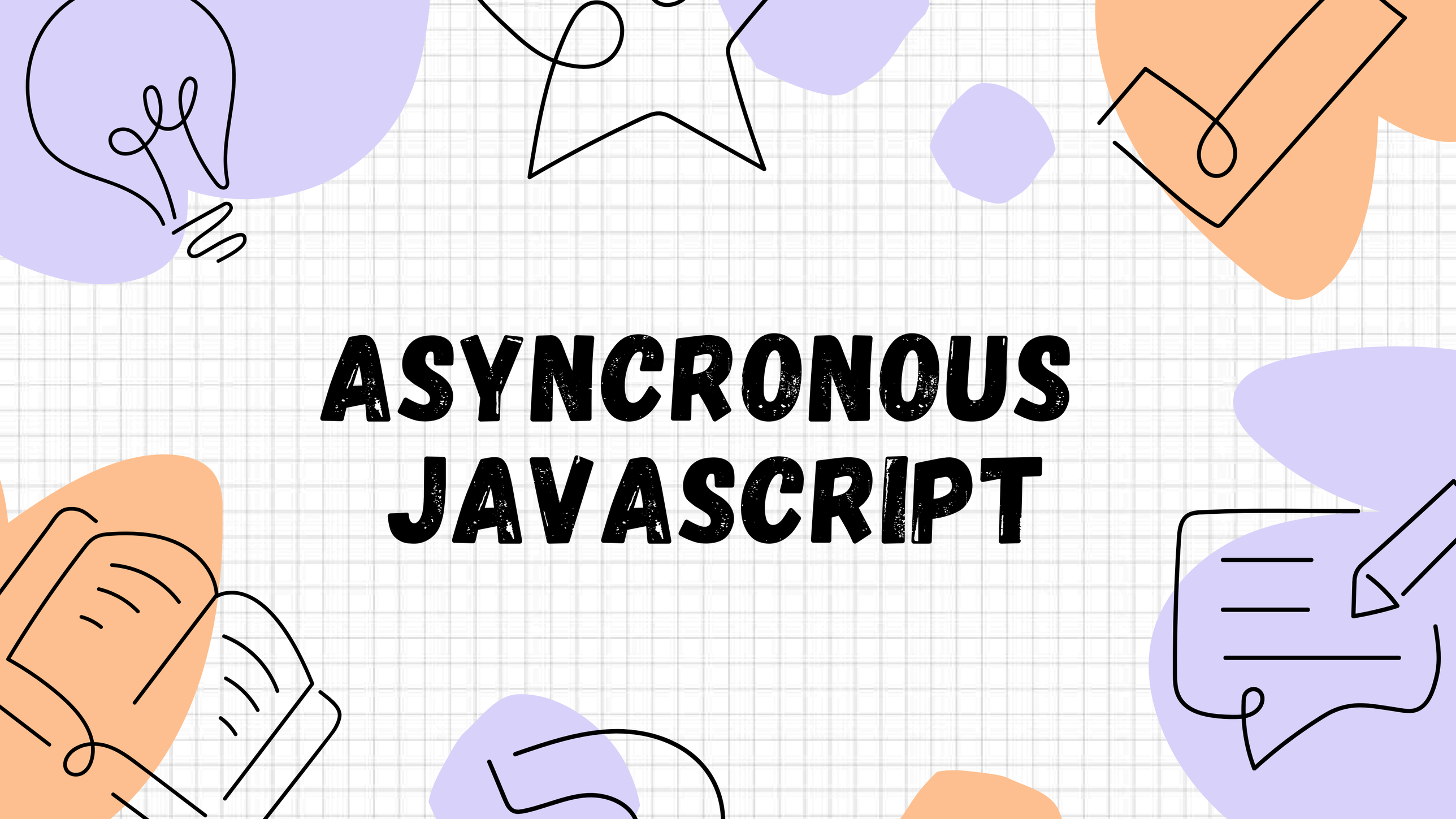
OBJECT METHODS:

```
//Functions
Object.keys(obj); // Returns an array of keys
Object.values(obj); // Returns an array of values
Object.entries(obj); // Returns an array of [key, value] pairs
```



JSON VS OBJECT




ASYNCRONOUS
JAVASCRIPT

SYNCRONOUS?

Fullstack Developer

Backend

↓

Frontend

↓

Integration

↓

Testing

↓

Deployment

ASYNCRONOUS?

Frontend Developer

Backend

↓

Frontend

↓

Integration

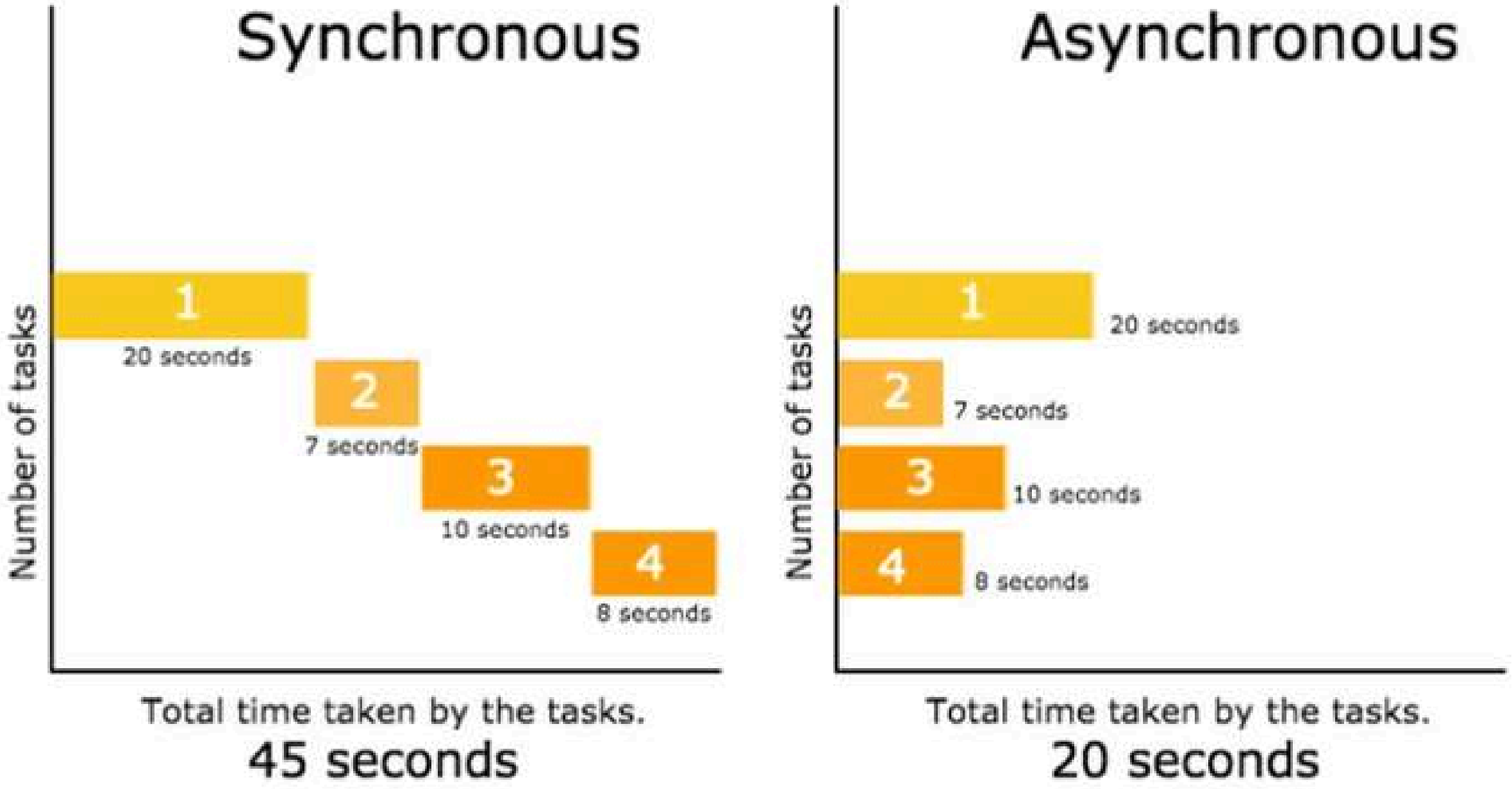
↓

Testing

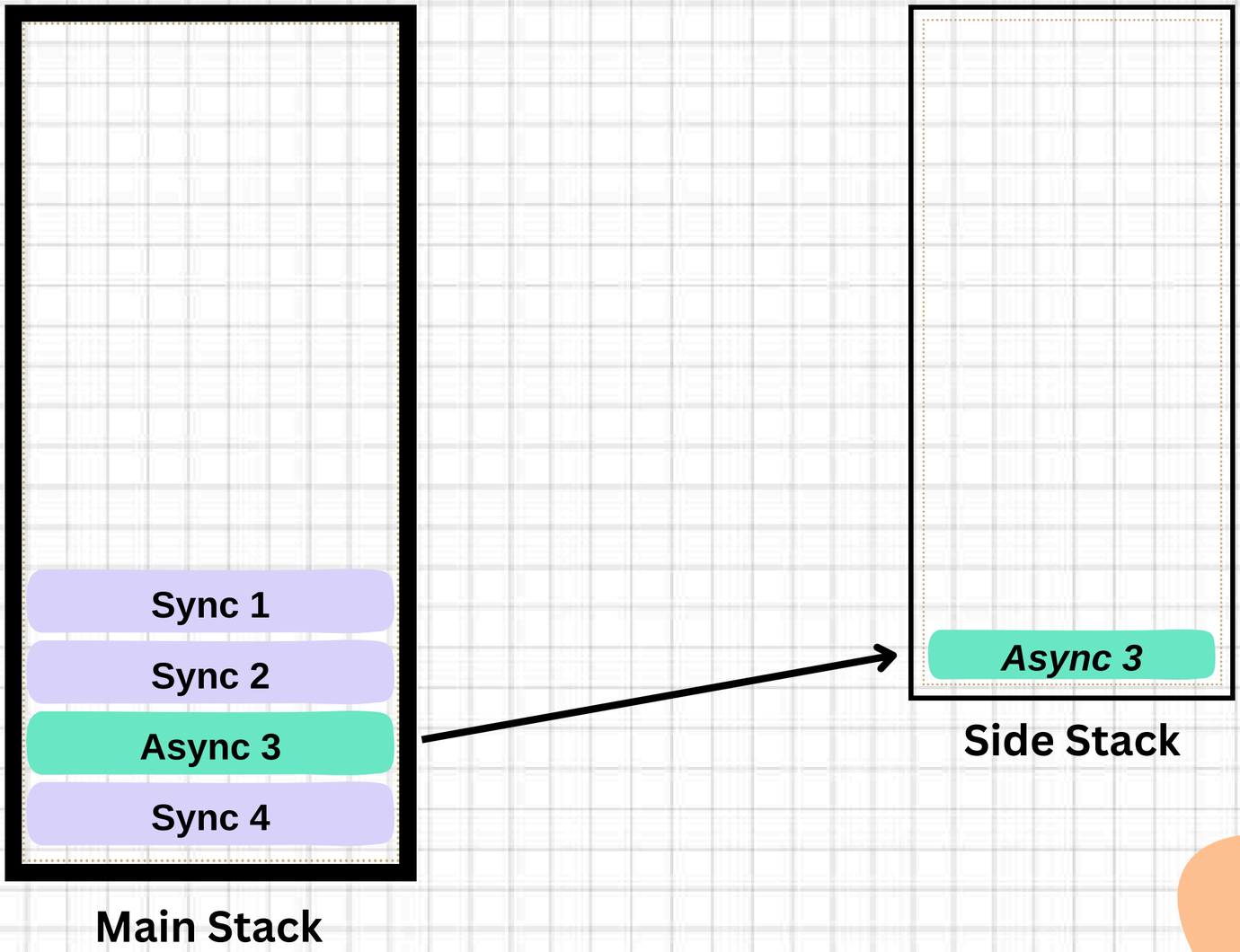
↓

Deployment

SYNCHRONOUS VS ASYNCHRONOUS

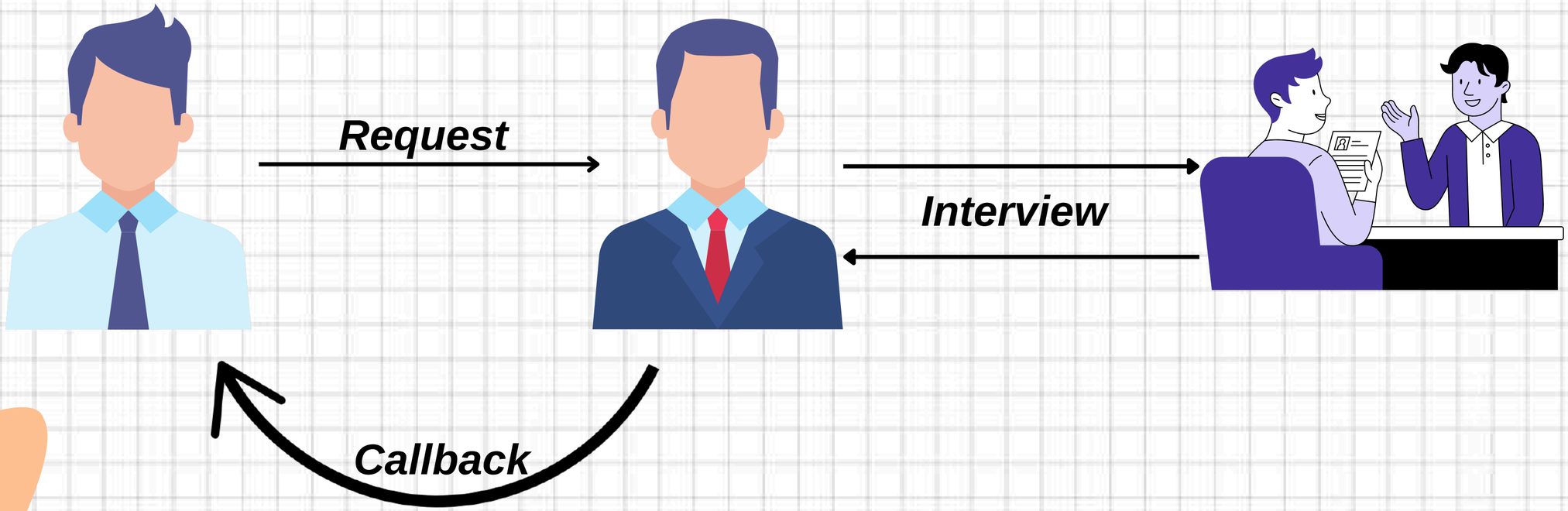


SYNCHRONOUS VS ASYNCHRONOUS

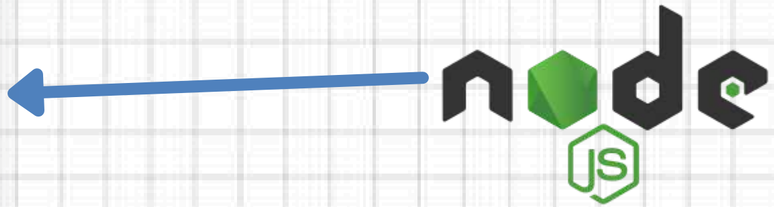


CALLBACKS

WHAT IS CALLBACK FUNCTION ?

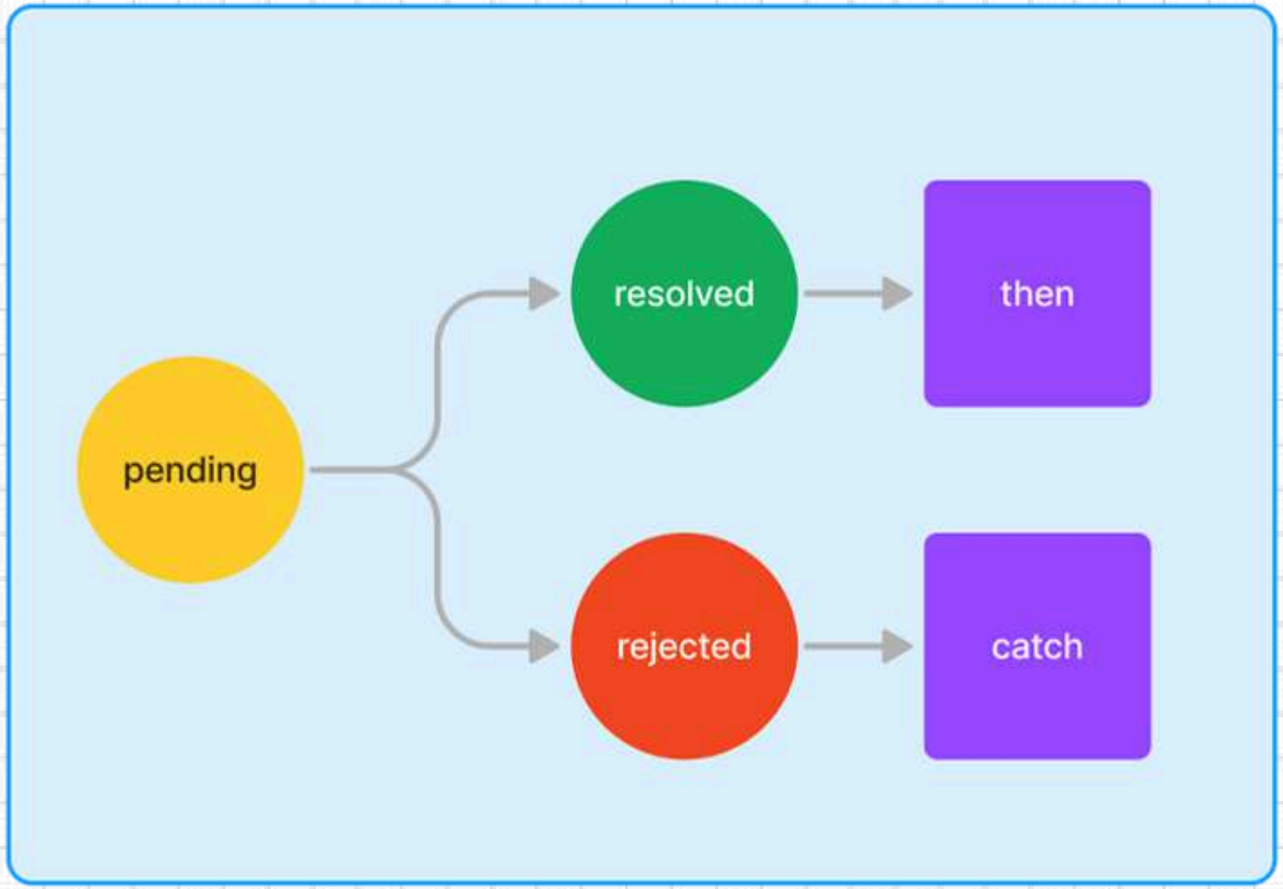


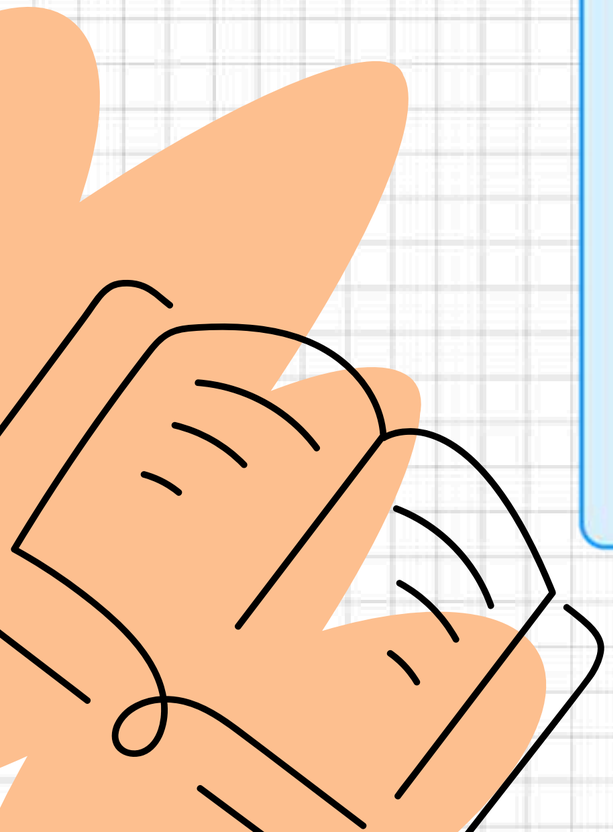
PROBLEMS WITH JAVASCRIPT

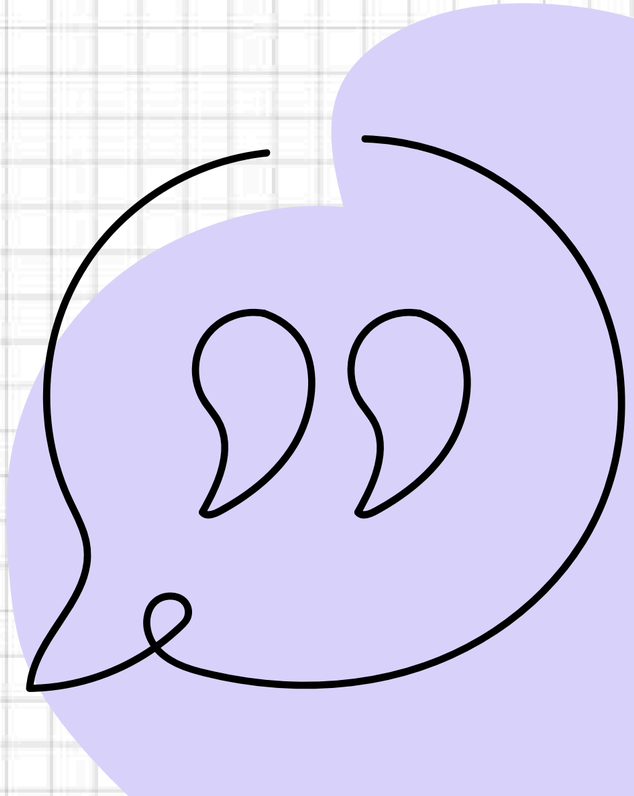


THE AKATSUKI CODING CLUB

PROMISES

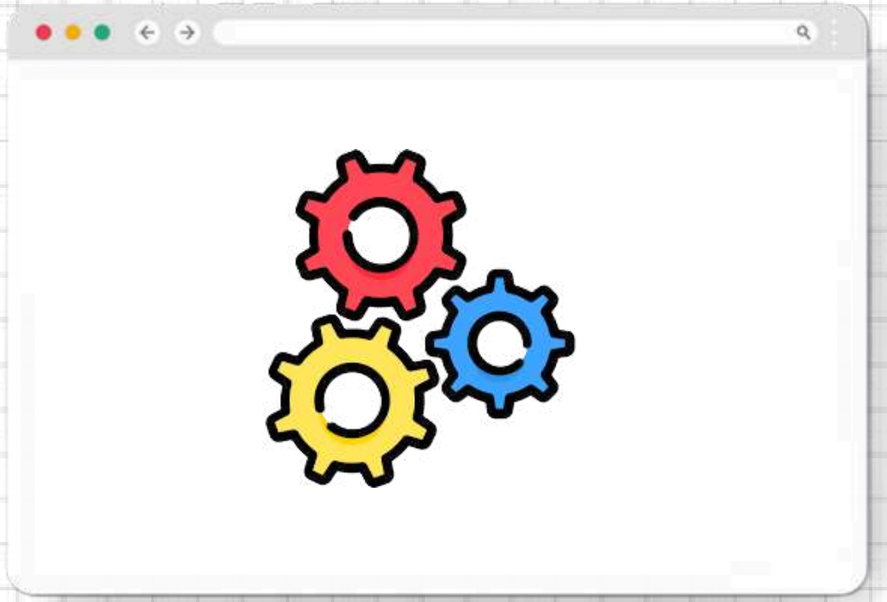


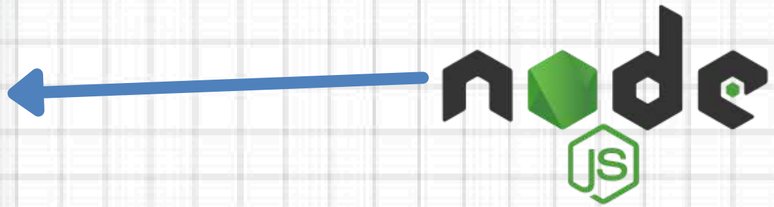


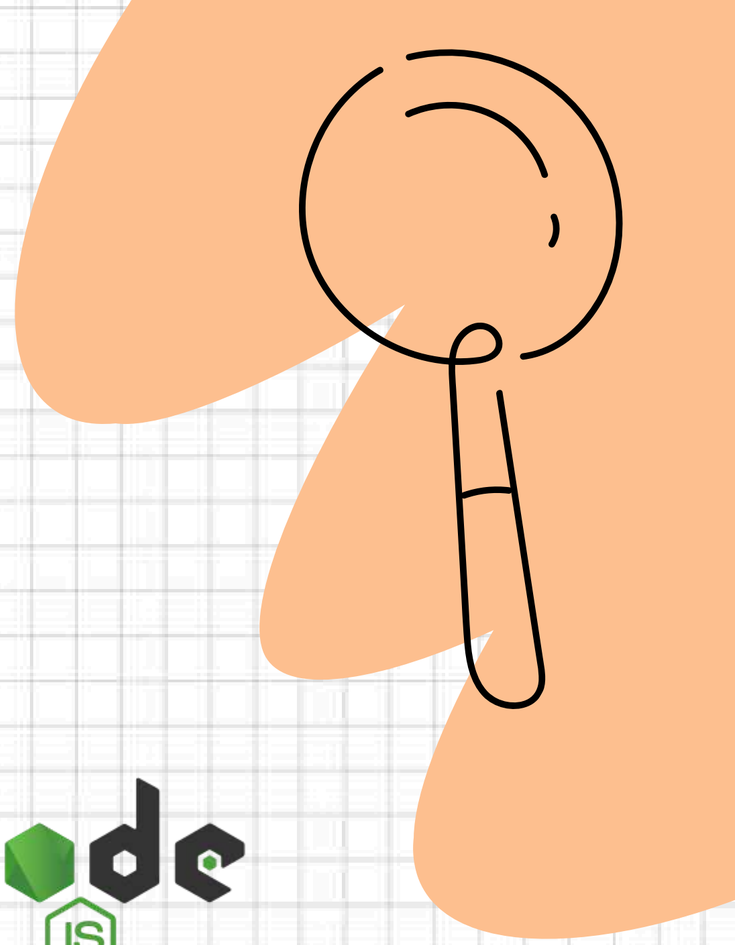


ASYNC/AWAIT

ERROR HANDLING (TRY N CATCH)

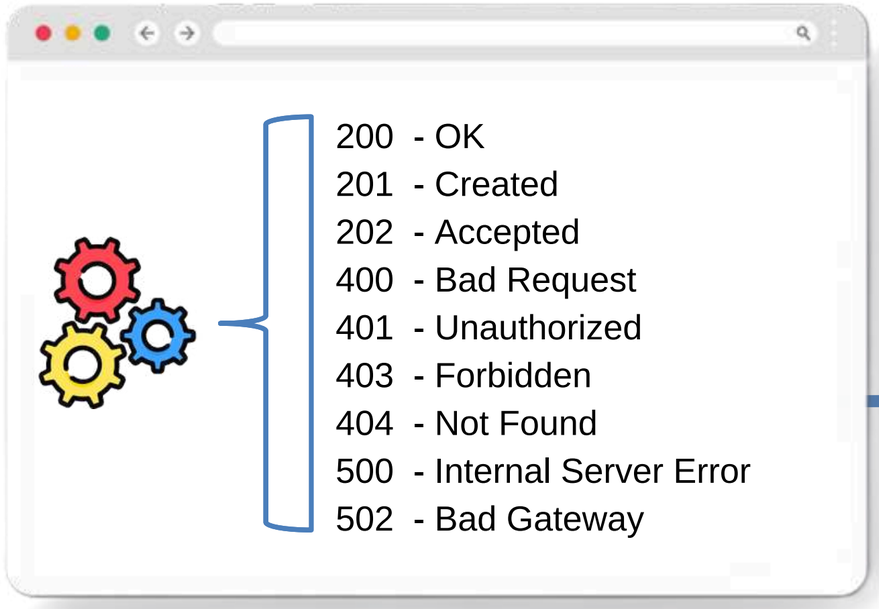




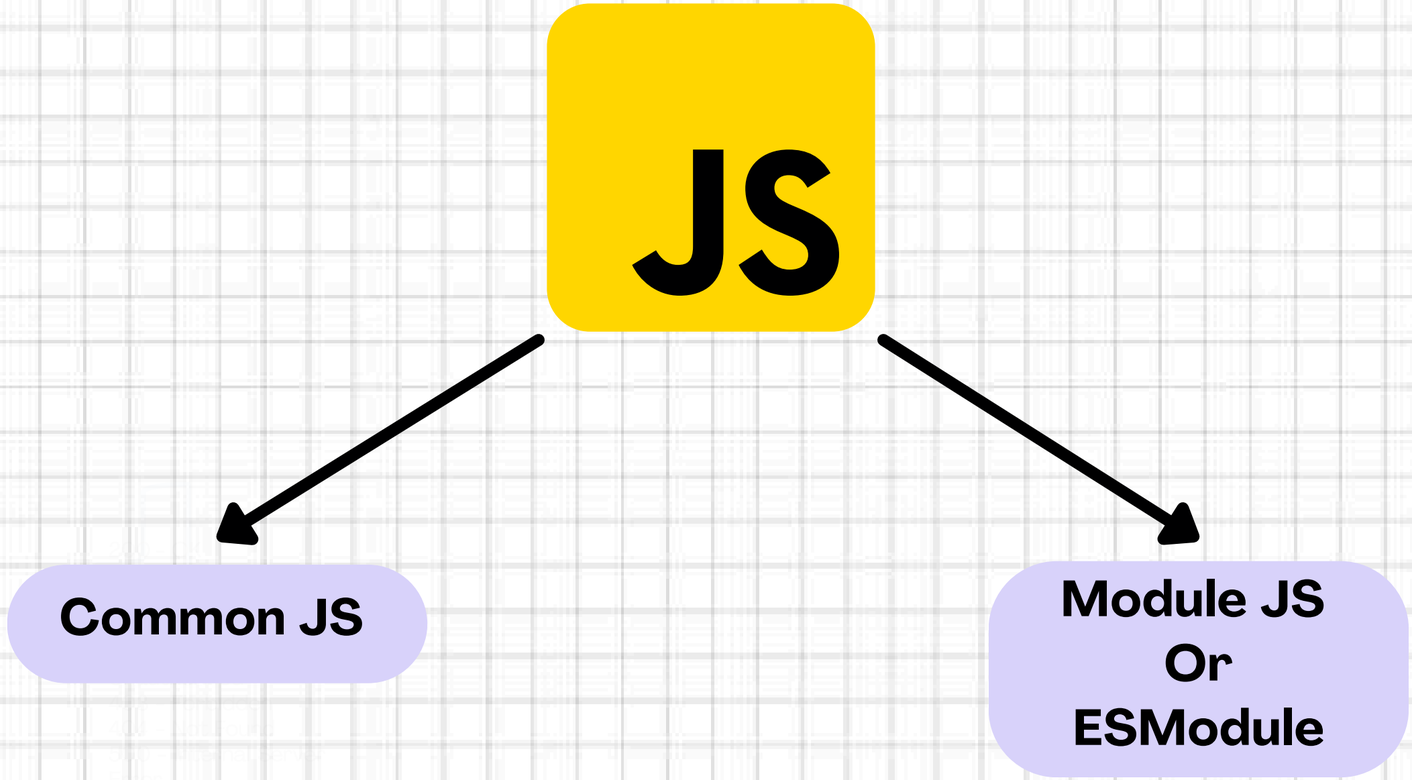


THE AKATSUKI CODING CLUB

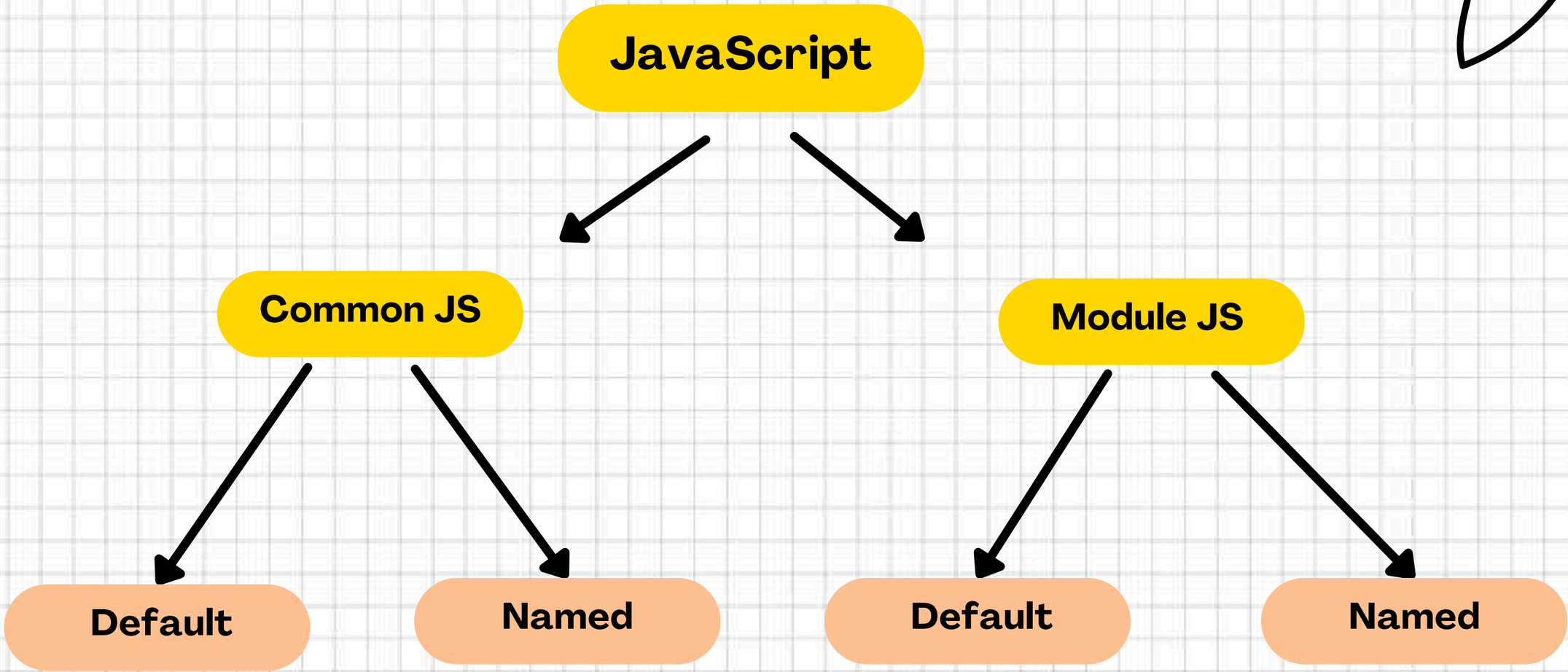
ERROR HANDLING (STATUS CODE)



TYPES OF JS



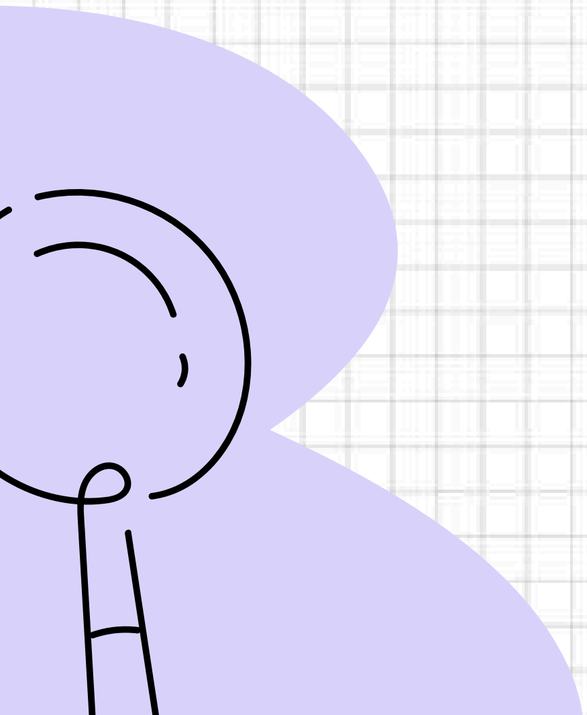
IMPORT AND EXPORT

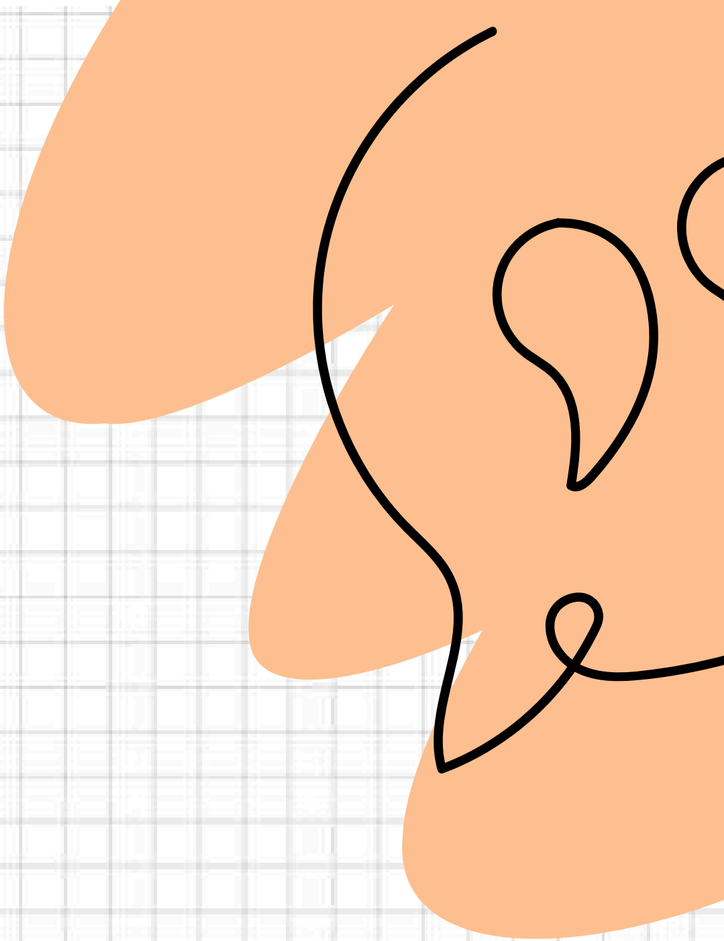


THE

AKATSUKI

CODING CLUB

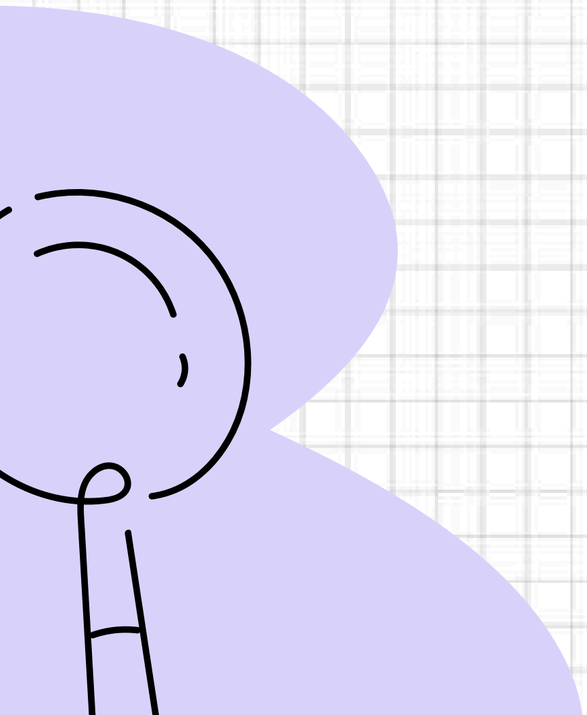


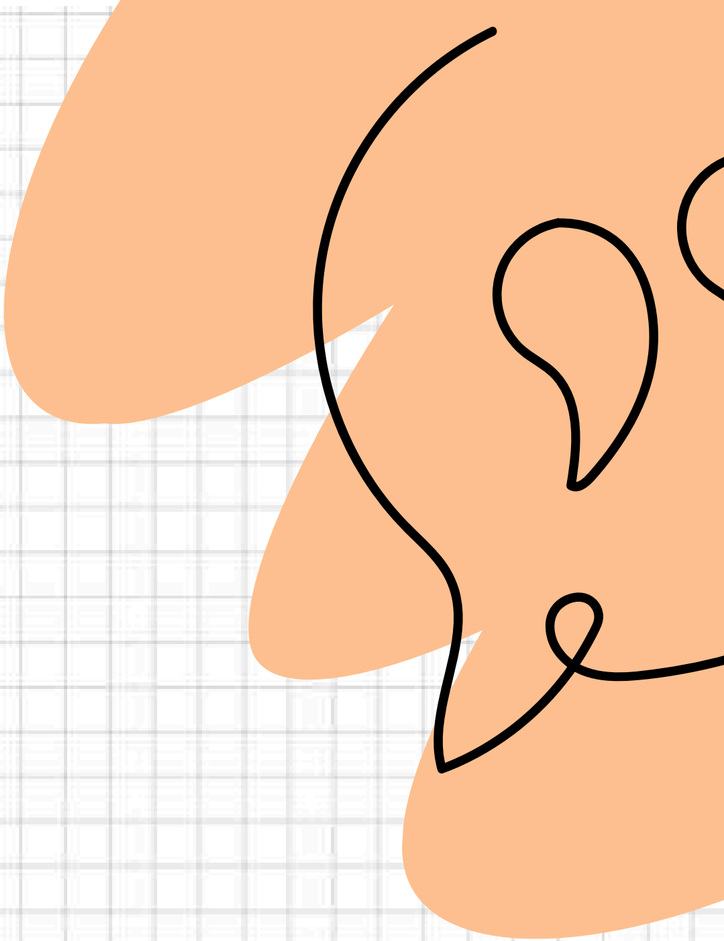


THE

AKATSUKI

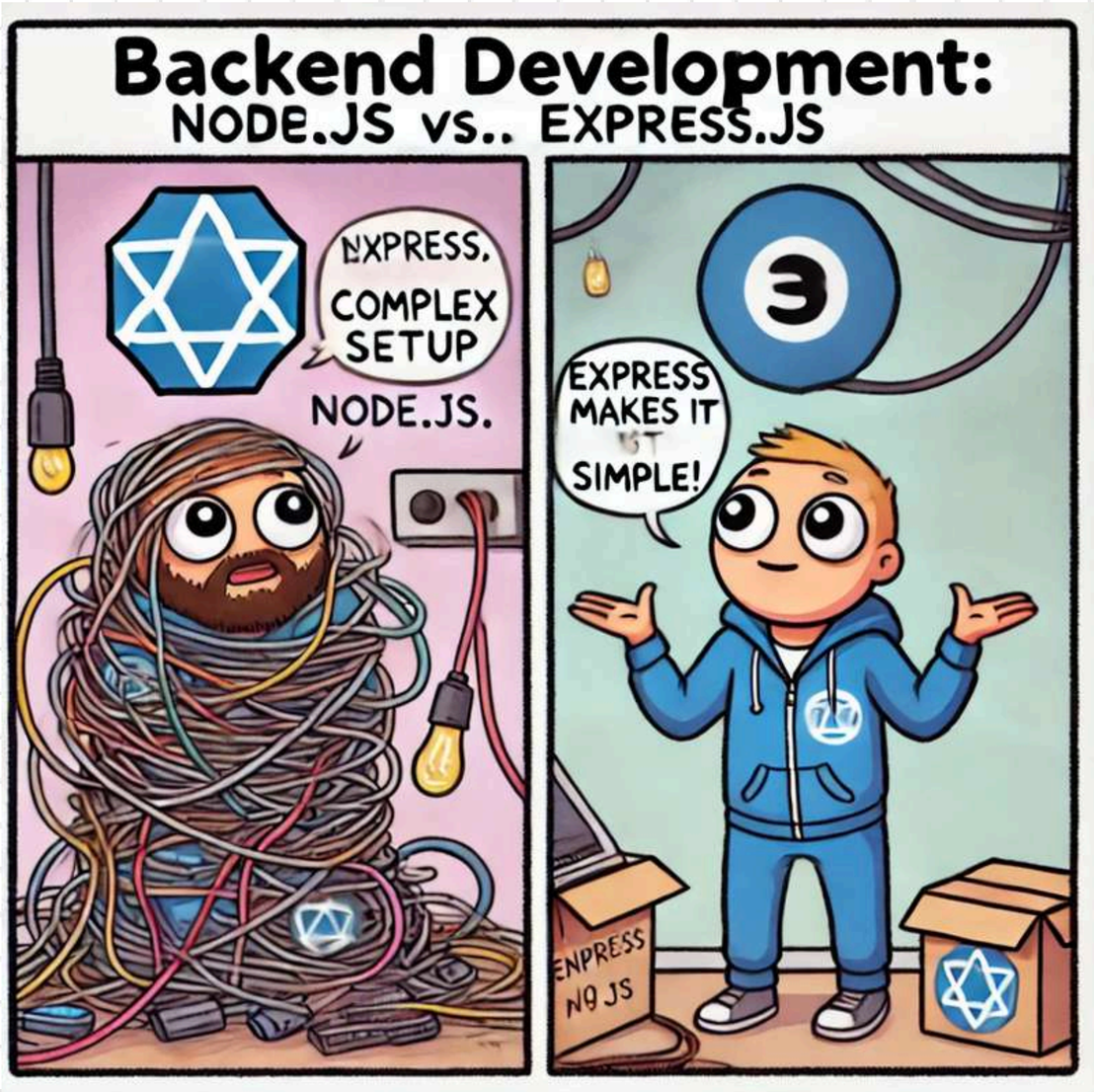
CODING CLUB



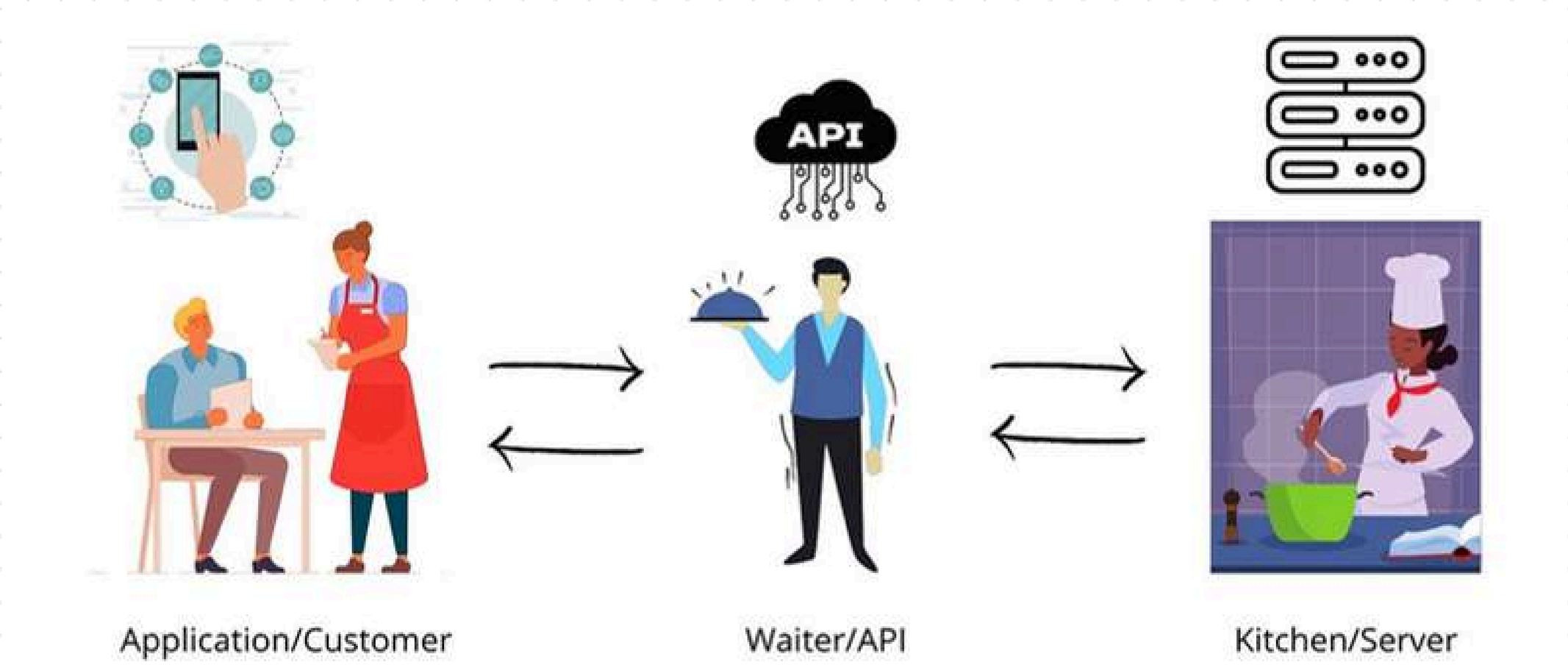


WHAT IS EXPRESS.JS?

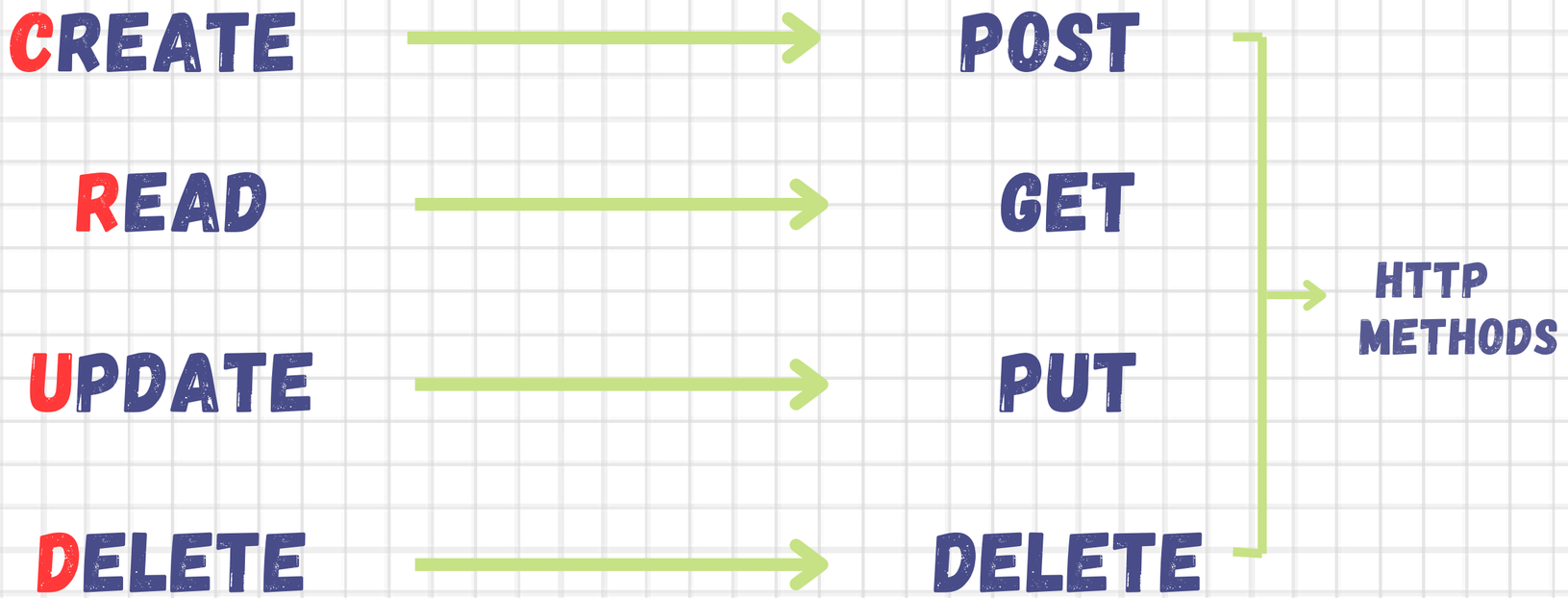
EXPRESS MAKES IT SIMPLE..



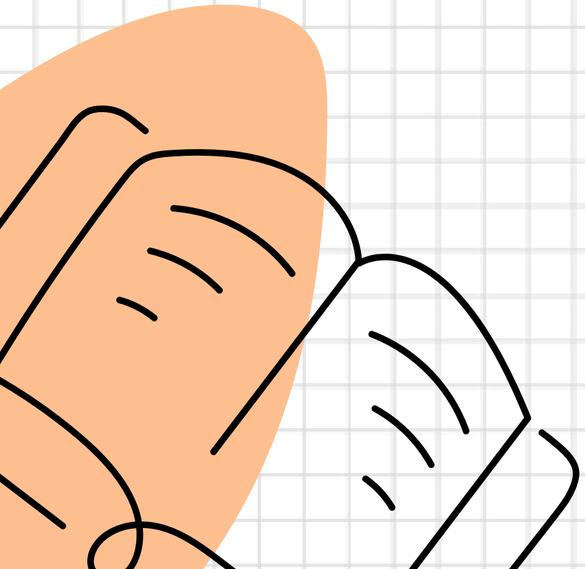
WHAT IS API?

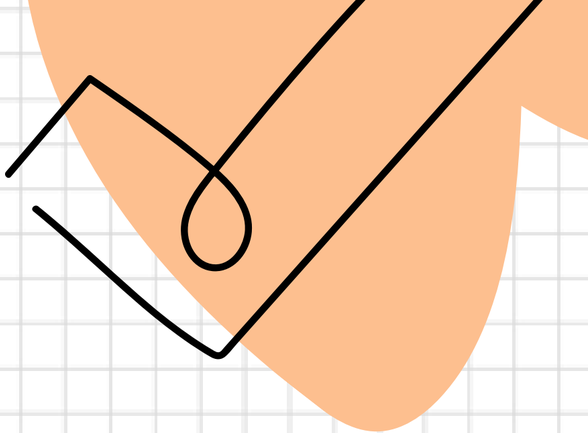


API REQUEST TYPES (HTTP METHODS)



LET'S CREATE AND TEST APIS...





Request

REQUEST LINE

Method: GET / POST / PUT / DELETE

Endpoint: /api/v1/getusers

HEADERS

Content-Type: application/json

Authorization: Bearer token

QUERY PARAMETERS

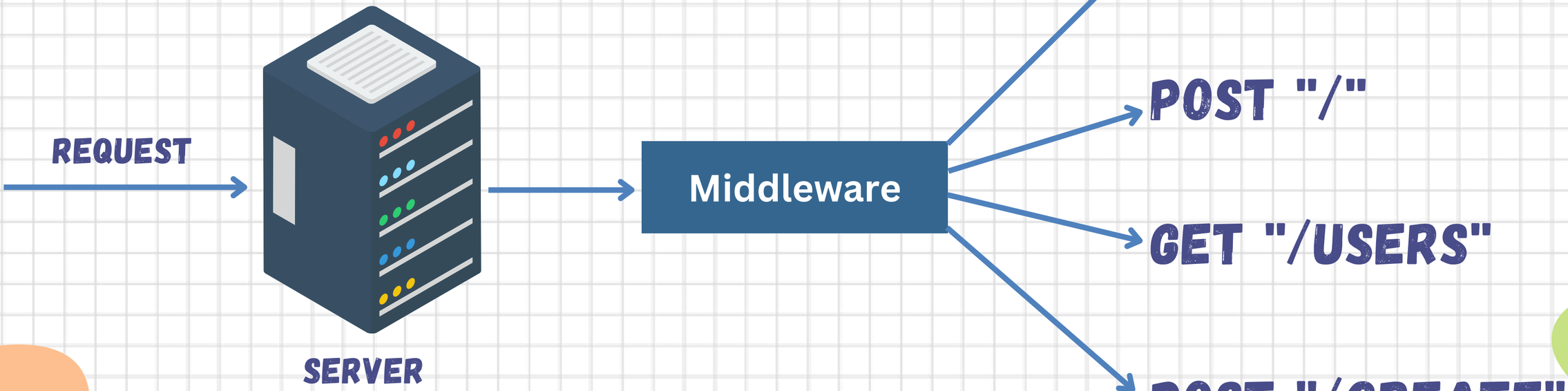
userId: 12345

sort: asc

BODY

```
{
    "name": "John Doe",
    "email": "john@example.com",
    "age": 30,
    "isActive": true,
    "roles": "Admin"
}
```

MIDDLEWARE



GET READY FOR MENTI QUIZ!

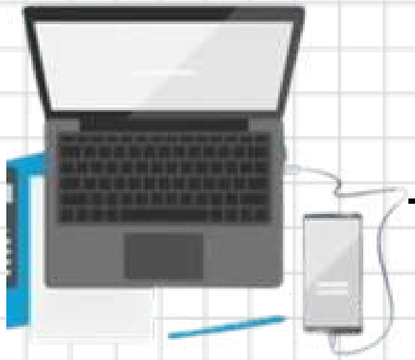

WELCOME TO
MONGODB
memegenerator.net

INTRODUCTION TO MONGODB

MongoDb is an open source, document-oriented No SQL database management system

mongoDB®

MORE ABOUT MONGODB

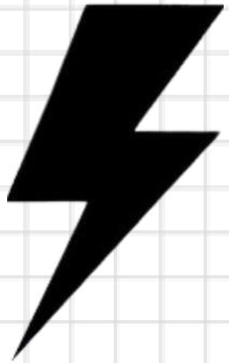


10gen

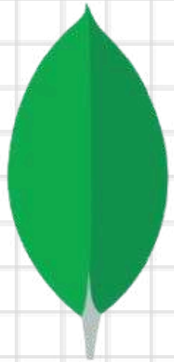
mongoDB

HUMONGOUS

MONGO

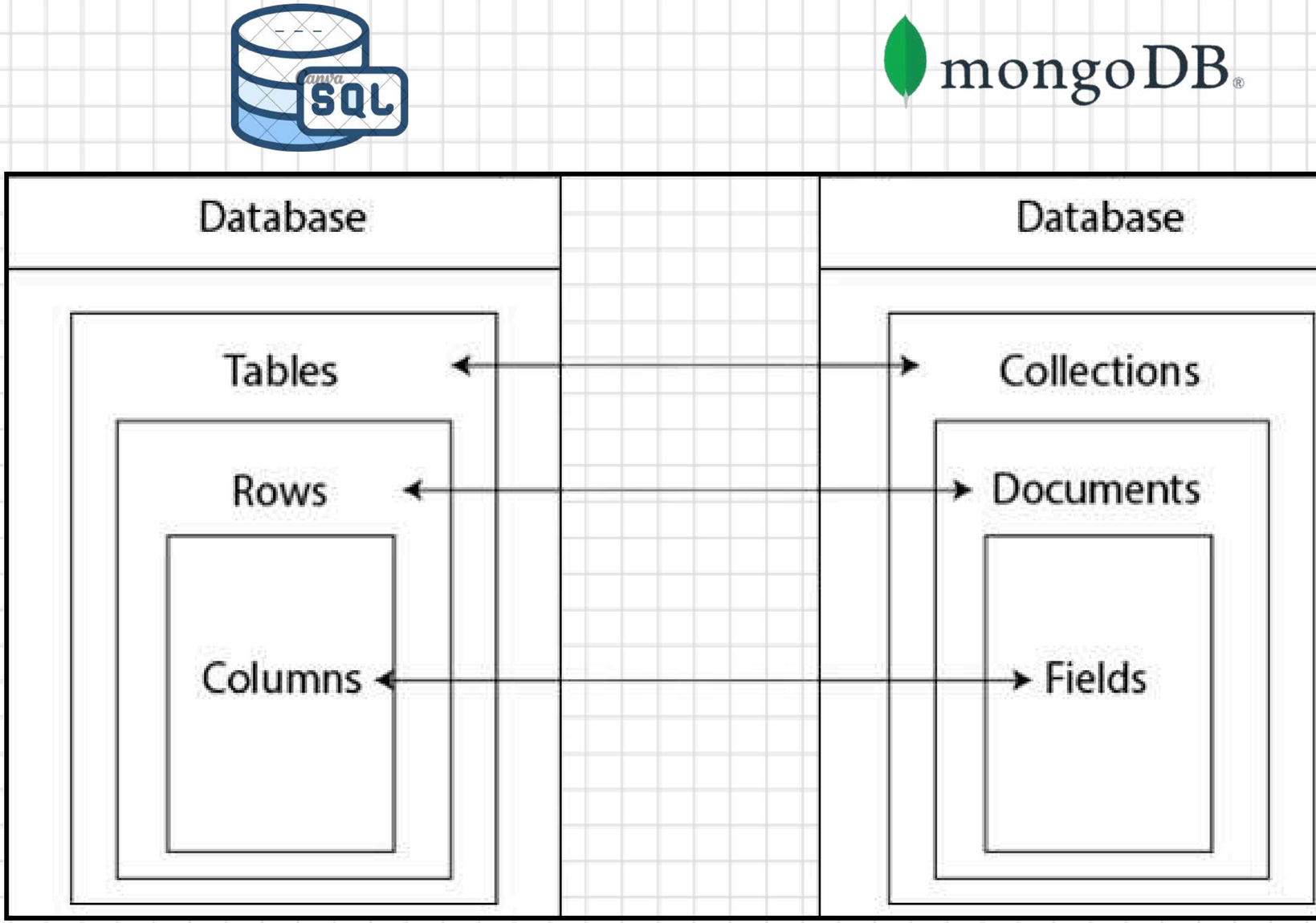


vs



mongoDB®

Database Structure



SQL

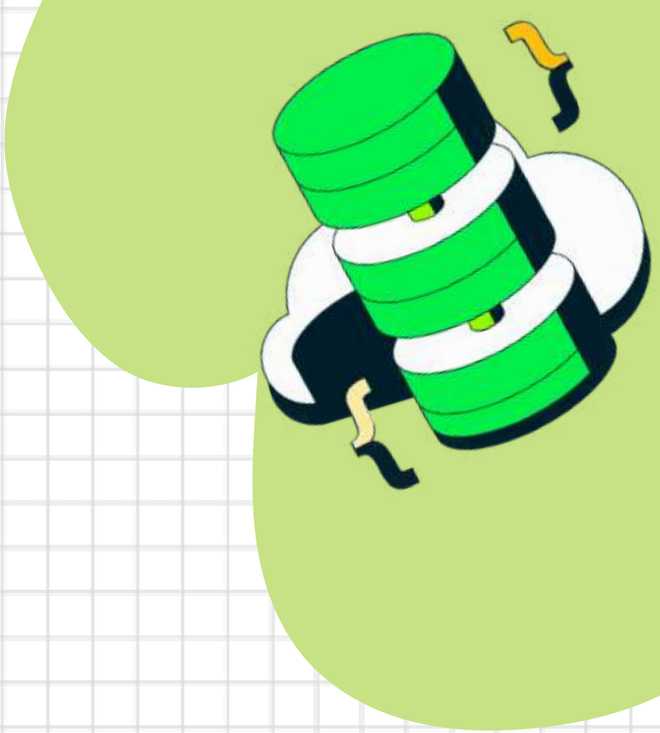
InstagramUsers

ID	Username	Name	Age
1.	wanderlust123	Alice Green	25
2.	techguru	John Doe	17
3.	fitlife	Tom White	68

NOSQL

```
{
    "InstagramUsers": [
        {
            "id": 1,
            "username": "wanderlust123",
            "name": "Alice Green",
            "age": 25
        },
        {
            "id": 2,
            "username": "techguru",
            "name": "John Doe",
            "age": 17
        },
        {
            "id": 3,
            "username": "fitlife",
            "name": "Tom White",
            "age": 68
        }
    ]
}
```

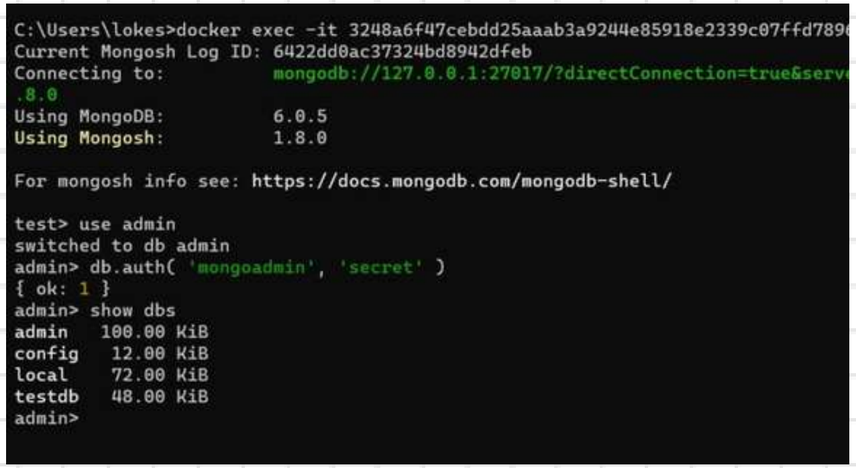
FEATURES..



- Flexible Schema Design
- Scalability and Performance
- Document Oriented Storage
- Dynamic Queries
- Open Source and Community

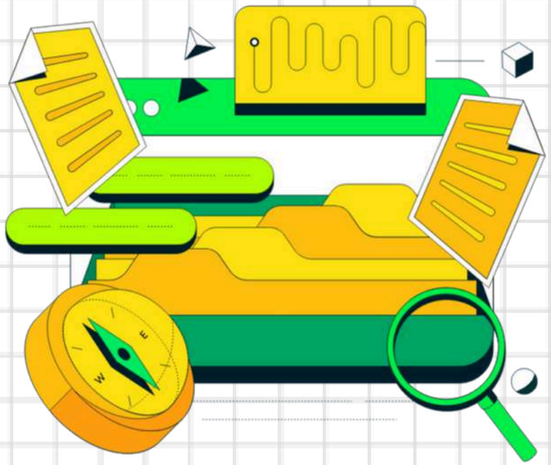
MONGODB COMPASS INSTALLATION...

DATABASE COMPONENTS



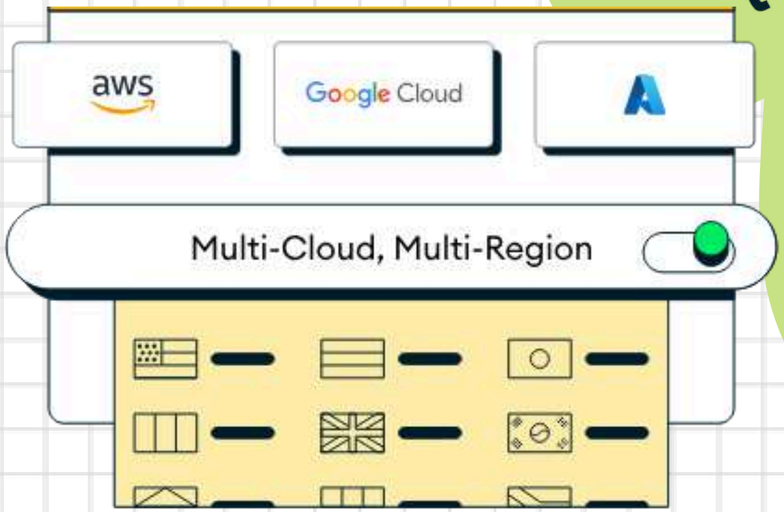
Local Database

e.g.MongoDB Shell



GUI Tool

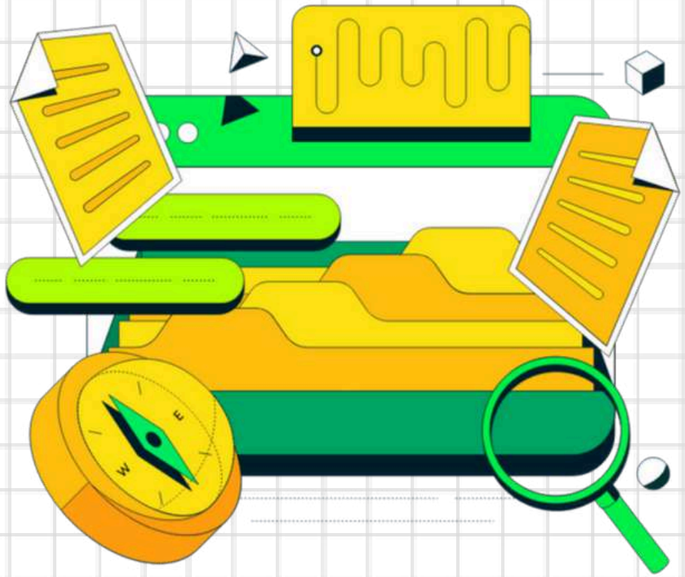
e.g. MongoDB Compass



Cloud Database

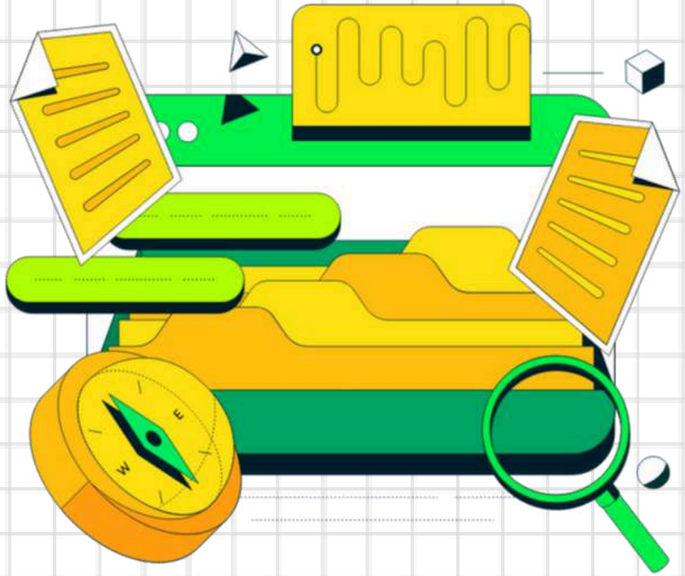
e.g. MongoDB Atlas

WHAT IS MONGODB ATLAS AND COMPASS ?



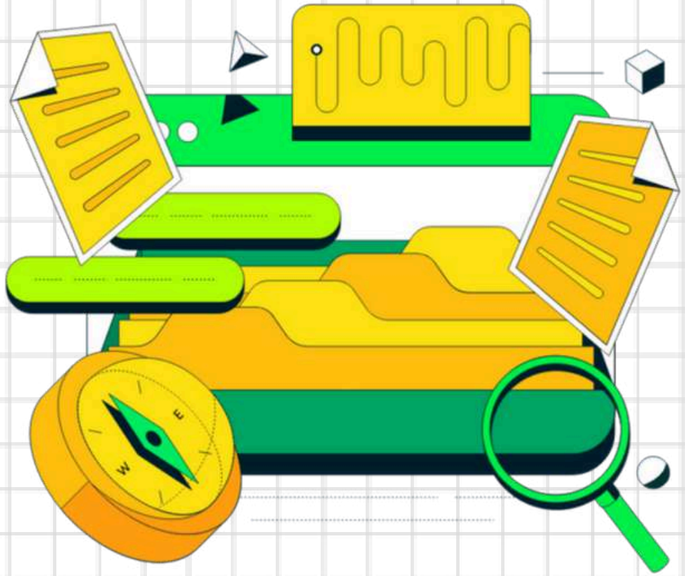
WHAT IS MONGODB ATLAS AND COMPASS ?

- Cloud Database Service



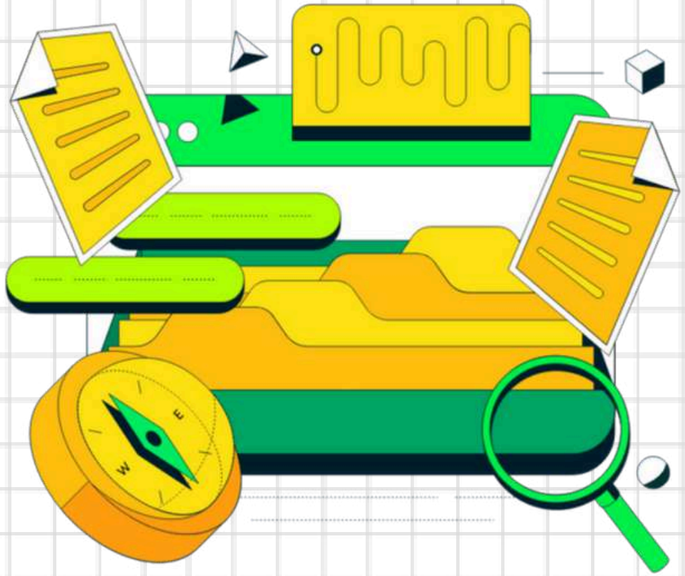
WHAT IS MONGODB ATLAS AND COMPASS ?

- Cloud Database Service
- Global Clusters



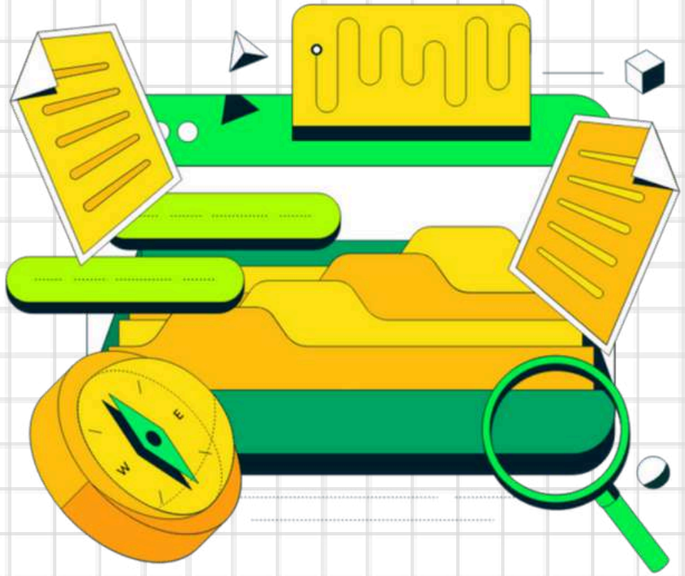
WHAT IS MONGODB ATLAS AND COMPASS ?

- Cloud Database Service
- Global Clusters
- Automated Backups and Recovery



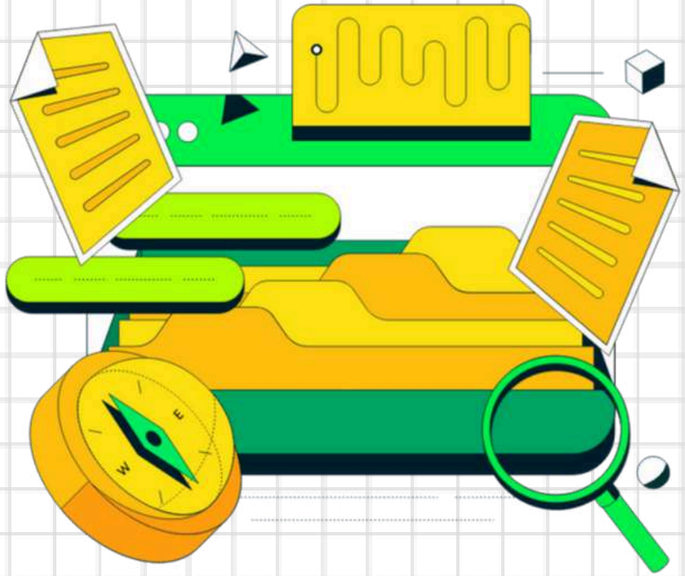
WHAT IS MONGODB ATLAS AND COMPASS ?

- Cloud Database Service
- Global Clusters
- Automated Backups and Recovery
- Graphical Interface



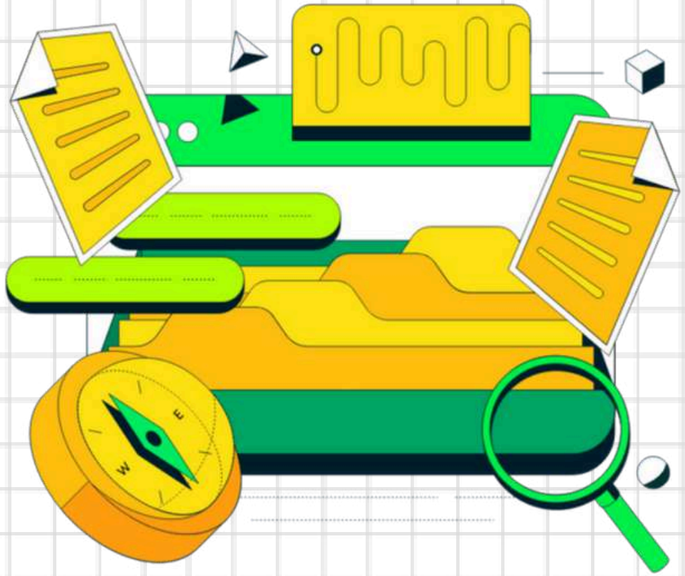
WHAT IS MONGODB ATLAS AND COMPASS ?

- Cloud Database Service
- Global Clusters
- Automated Backups and Recovery
- Graphical Interface
- Document Editor

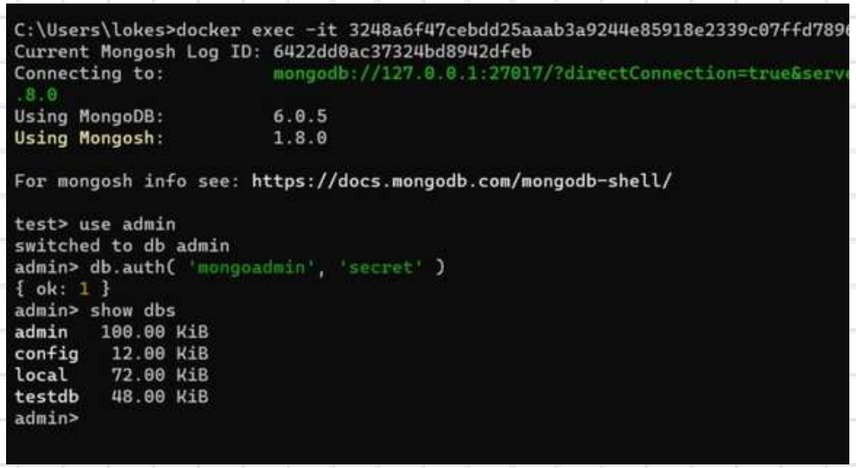


WHAT IS MONGODB ATLAS AND COMPASS ?

- Cloud Database Service
- Global Clusters
- Automated Backups and Recovery
- Graphical Interface
- Document Editor
- Connection Manager

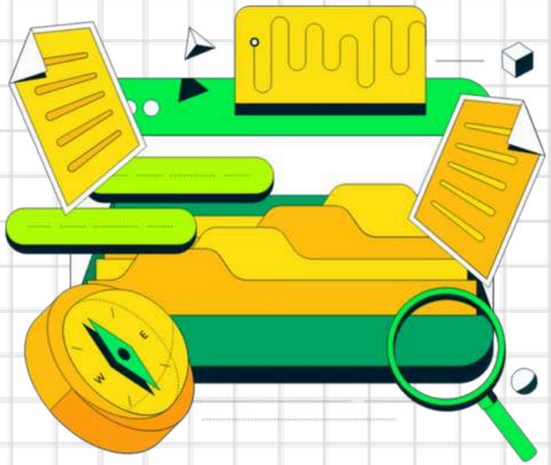


DATABASE COMPONENTS



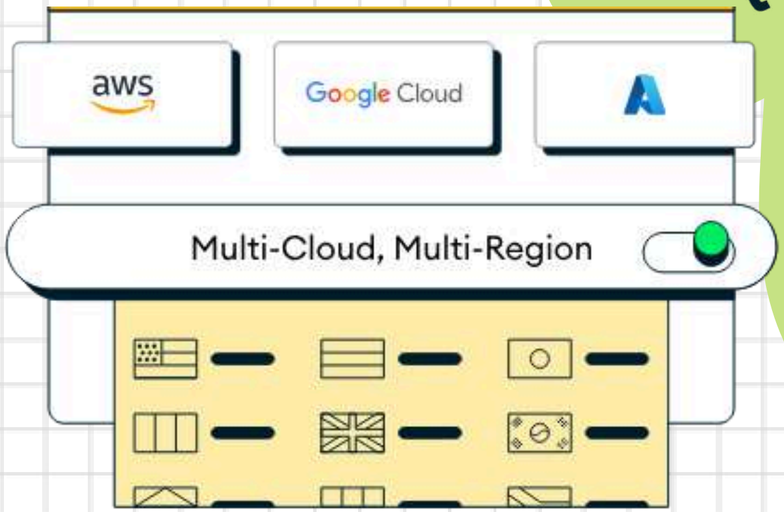
Local Database

e.g.MongoDB Shell



GUI Tool

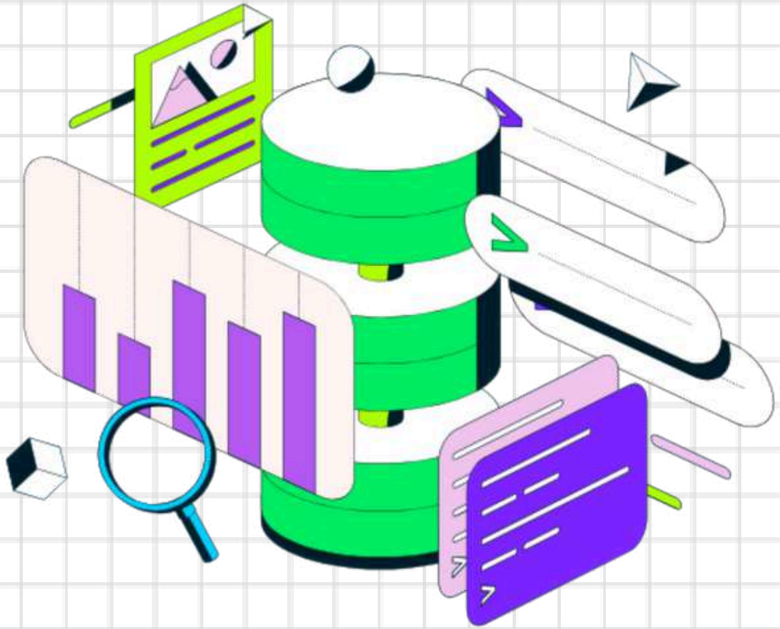
e.g. MongoDB Compass

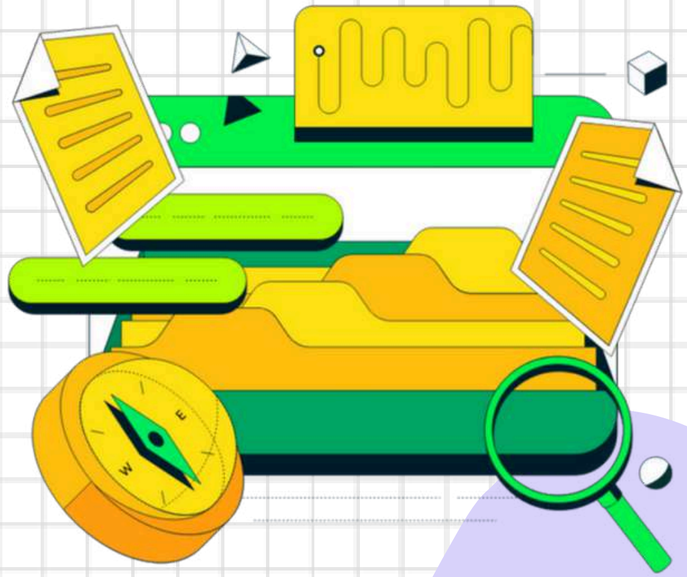


Cloud Database

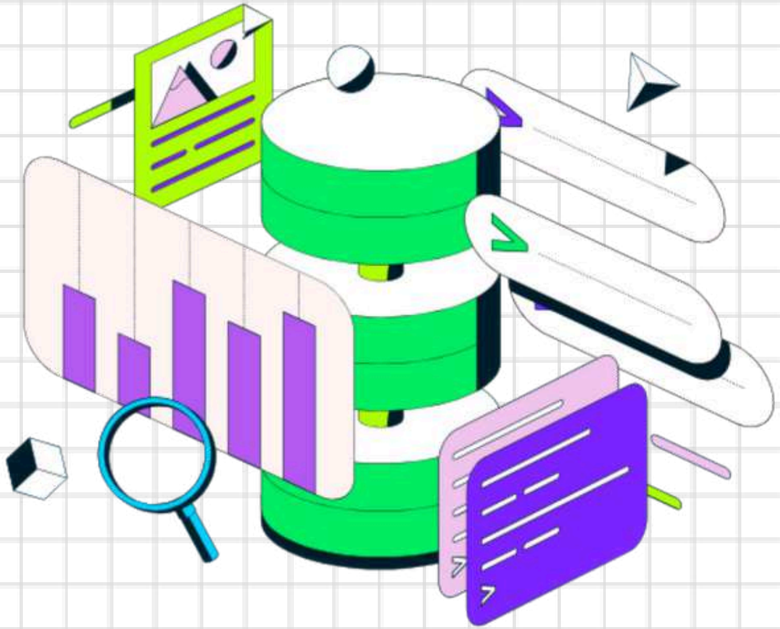
e.g. MongoDB Atlas

CREATING A CLUSTER AND CONNECTING WITH MONGODB COMPASS...

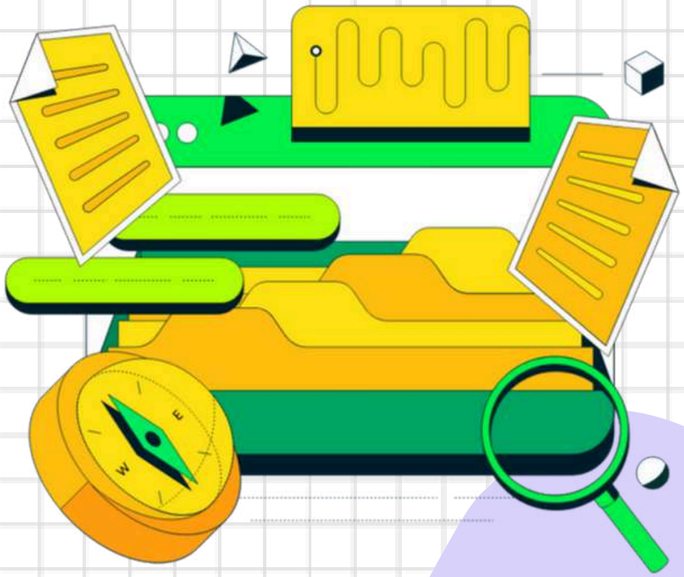




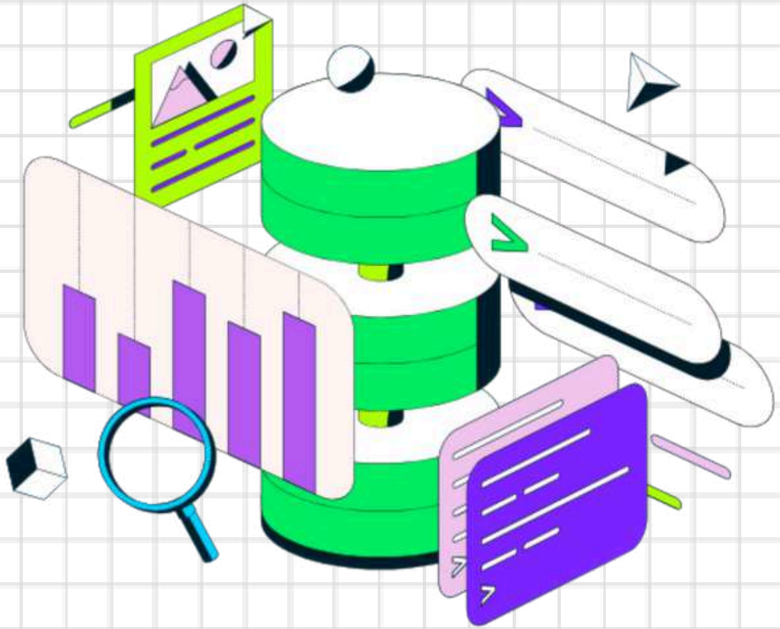
CREATING A CLUSTER AND CONNECTING WITH MONGODB COMPASS...



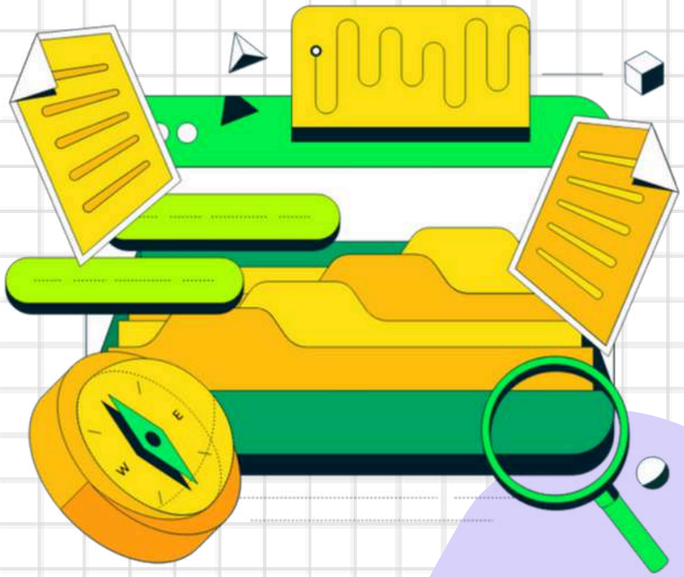
Connecting...

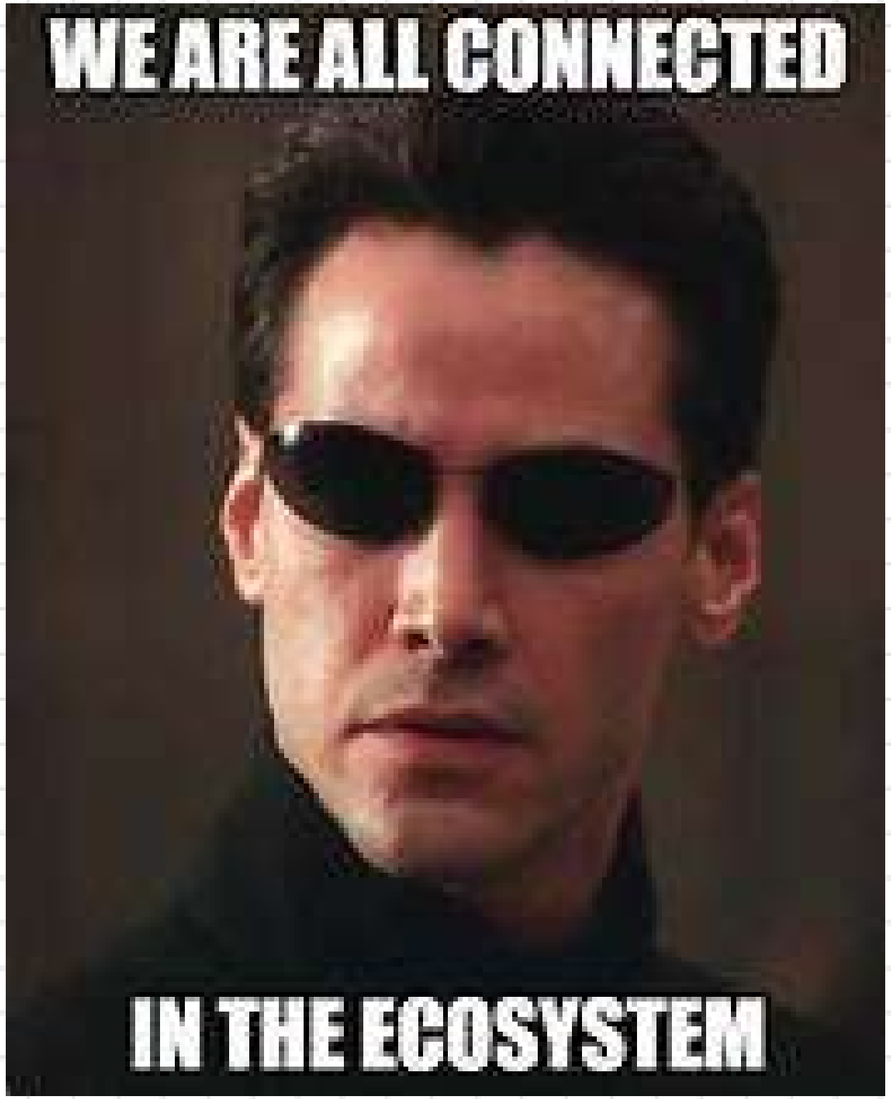


CREATING A CLUSTER AND CONNECTING WITH MONGODB COMPASS...



Connected....



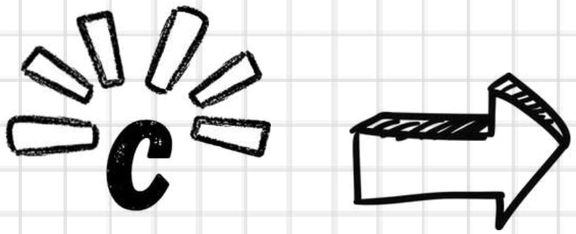

WE ARE ALL CONNECTED
IN THE ECOSYSTEM

MANAGING DATABASES AND COLLECTIONS :

- show dbs;
- use <database-name>;
- db.createCollection(‘<collection-name>’);
- show collections;
- db.<collection-name>.drop();
- db.dropDatabase();

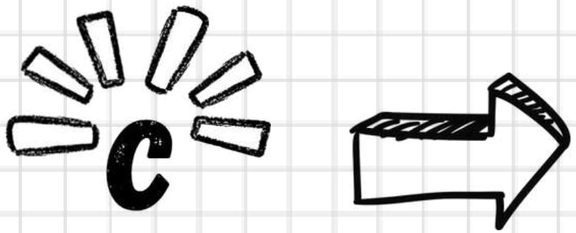
CRUD OPERATIONS

CRUD OPERATIONS

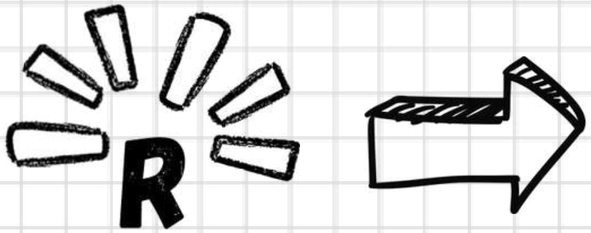


THE CREATE OPERATION IS USED TO INSERT NEW DOCUMENTS IN THE MONGODB DATABASE.

CRUD OPERATIONS

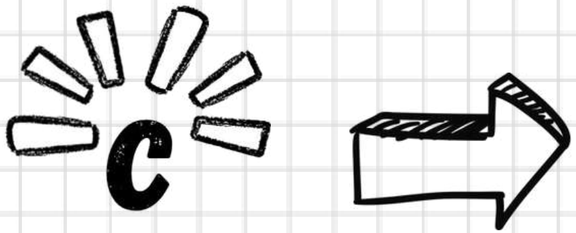


THE CREATE OPERATION IS USED TO INSERT NEW DOCUMENTS IN THE MONGODB DATABASE.

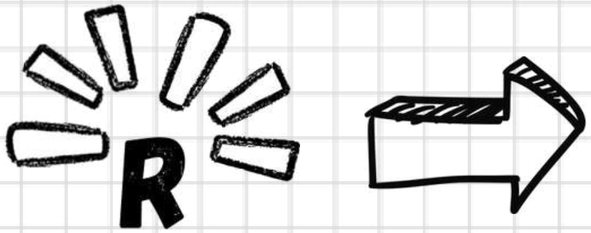


THE READ OPERATION IS USED TO QUERY A DOCUMENT IN THE DATABASE.

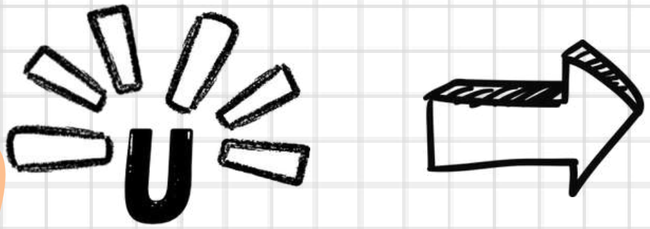
CRUD OPERATIONS



THE CREATE OPERATION IS USED TO INSERT NEW DOCUMENTS IN THE MONGODB DATABASE.

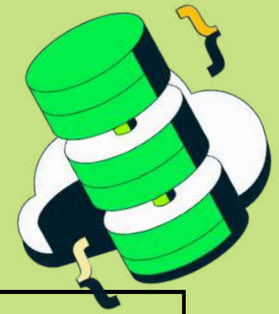


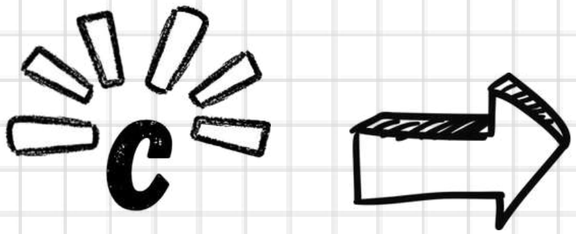
THE READ OPERATION IS USED TO QUERY A DOCUMENT IN THE DATABASE.



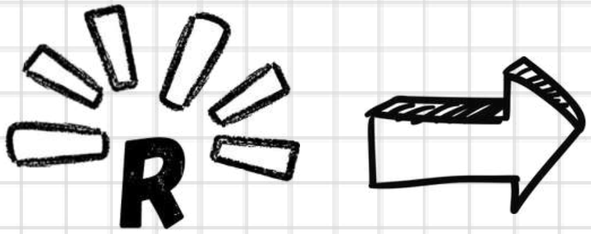
THE UPDATE OPERATION IS USED TO MODIFY EXISTING DOCUMENTS IN THE DATABASE.

CRUD OPERATIONS

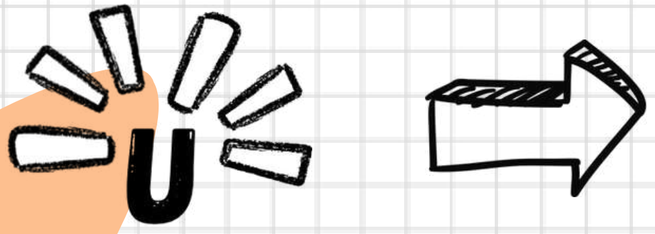




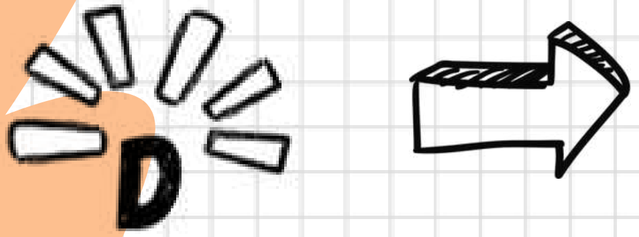
THE CREATE OPERATION IS USED TO INSERT NEW DOCUMENTS IN THE MONGODB DATABASE.



THE READ OPERATION IS USED TO QUERY A DOCUMENT IN THE DATABASE.



THE UPDATE OPERATION IS USED TO MODIFY EXISTING DOCUMENTS IN THE DATABASE.



THE DELETE OPERATION IS USED TO REMOVE DOCUMENTS IN THE DATABASE.

CREATE(INSERT DOCUMENT) :

CREATE(INSERT DOCUMENT) :

INSERTONE(); ⟶

```
db.<collection-name>.insertOne({
        field1: value1,
        field2: value2,
        ...
});
```

CREATE(INSERT DOCUMENT) :

INSERTONE();

```
db.<collection-name>.insertOne({
      field1: value1,
      field2: value2,
      ...
});
```

INSERTMANY();

```
db.<collection-name>.insertMany([
      { field1: value1, field2: value2, ... },
      { field1: value1, field2: value2, ... },
      // ...
]);
```

READ DOCUMENT :

READ DOCUMENT :

FIND();

```
db.collection_name.find({ key: value })
```

READ DOCUMENT :

FIND(); ⟶

```
db.collection_name.find({ key: value })
```

FINDONE(); ⟶

```
db.collection_name.findOne({ key: value })
```

COMPARISON OPERATORS :

$eq	$ne	$gt	$gte
$lt	$lte	$in	$nin

UPDATE DOCUMENT :

UPDATE DOCUMENT :

updateOne();

```
db.collectionName.updateOne(
      { filter },
      { $set: { existingField: newValue, newField: "new value", // ... }, }
);
```

UPDATE DOCUMENT :

updateOne();

```
db.collectionName.updateOne(
      { filter },
      { $set: { existingField: newValue, newField: "new value", // ... }, }
);
```

updateMany();

```
db.collectionName.updateMany(
      { filter },
      { $set: { existingField: newValue, // ... }, }
);
```

REMOVING AND UPDATING FIELDS :

REMOVING AND UPDATING FIELDS :

$unset:

```
db.collectionName.updateOne( { filter }, { $unset: { fieldName: 1 } } );
```

REMOVING AND UPDATING FIELDS :

$unset:

```
db.collectionName.updateOne( { filter }, { $unset: { fieldName: 1 } } );
```

$rename:

```
db.collectionName.updateOne(
      { filter },
      { $rename: { oldFieldName: "newFieldName" } }
  );
```

DELETE DOCUMENT :

DELETE DOCUMENT :

deleteOne();

```
db.collectionName.deleteOne({ filter });
```

DELETE DOCUMENT :

deleteOne();

```
db.collectionName.deleteOne({ filter });
```

deleteMany();

```
db.sales.deleteMany({ price: 55 });
```


YOU GET A DATABASE! AND
YOU GET A DATABASE!
EVERYONE GETS A
DATABASE!!!!

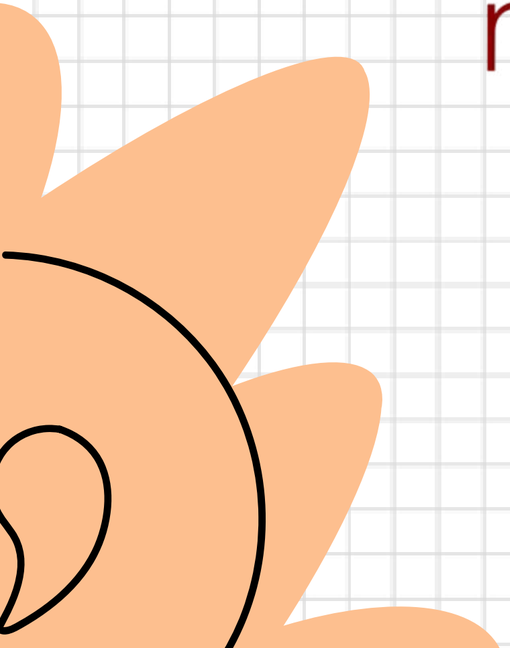
WHAT IS MONGOOSE?

- Object Data Modeling (ODM) library.
- Helps you interact with Node js (objects) and MongoDB database (json)

mongoose

WHAT MONGOOSE DO

WHAT ARE SCHEMAS IN



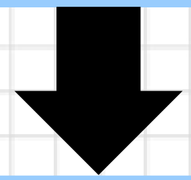
```
const mongoose = require('mongoose');

const user=new mongoose.Schema({
    name:{
        type: String,
        require: true,
    },
    email:{
        type: String,
        require: true,
        unique:true,
    },
    password:{
        type: String,
        require: true,
    },
    isAdmin:{
        type: Boolean,
        default: false,
    },
});
```

WHAT ARE MODEL IN

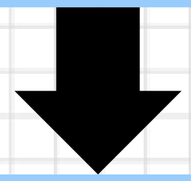

```
const mongoose = require('mongoose');

const user = new mongoose.Schema({
    name:{ ···
    },
    email:{ ···
    },
    password:{ ···
    },
    isAdmin:{ ···
    },
});

const User = mongoose.model('User', user);

module.exports = User;
```

STEPS TO CREATE COLLECTION USING MONGOOSE

Define Schema

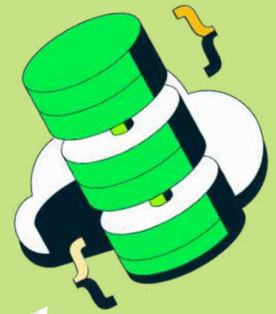


Create it’s Model

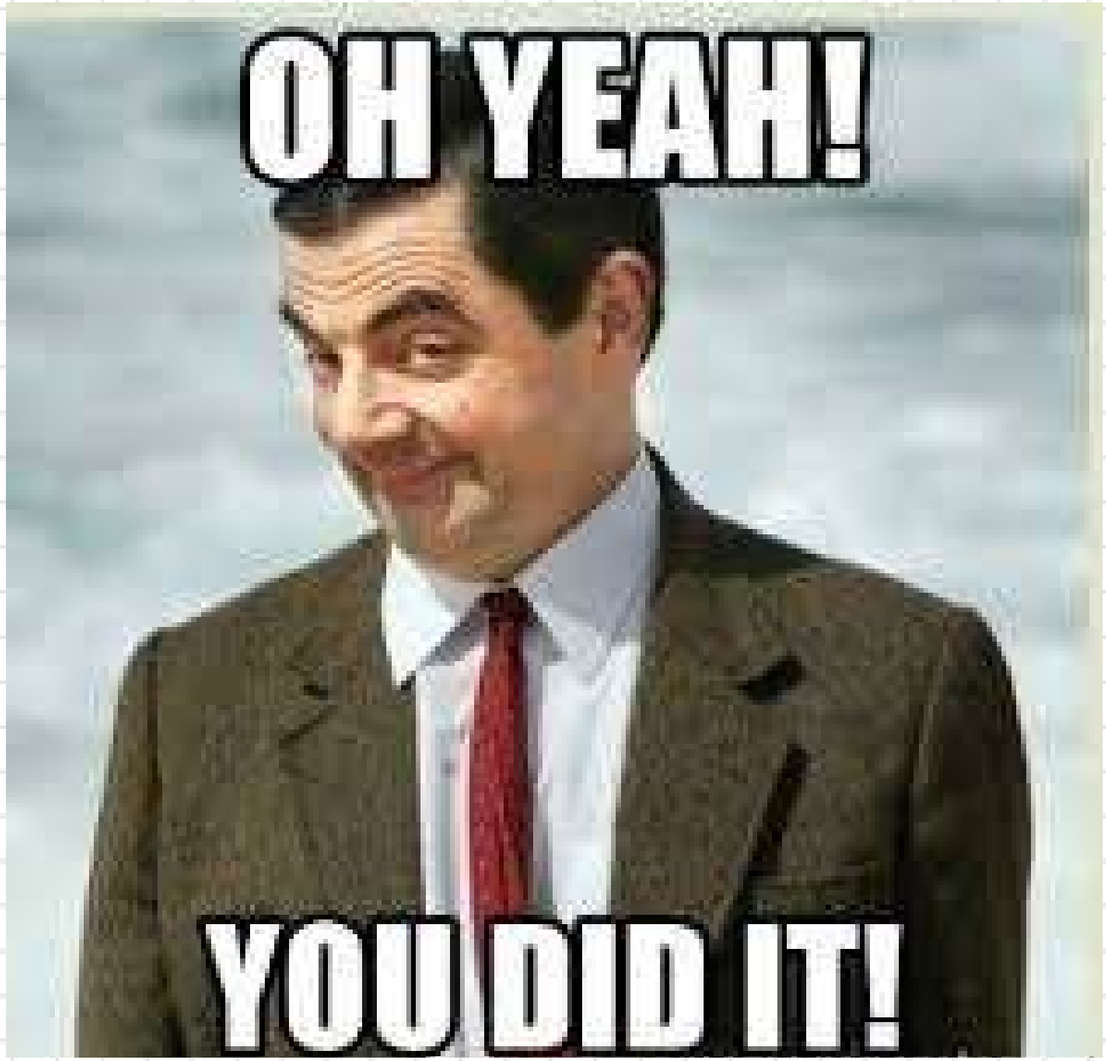


Export model
(not schema)

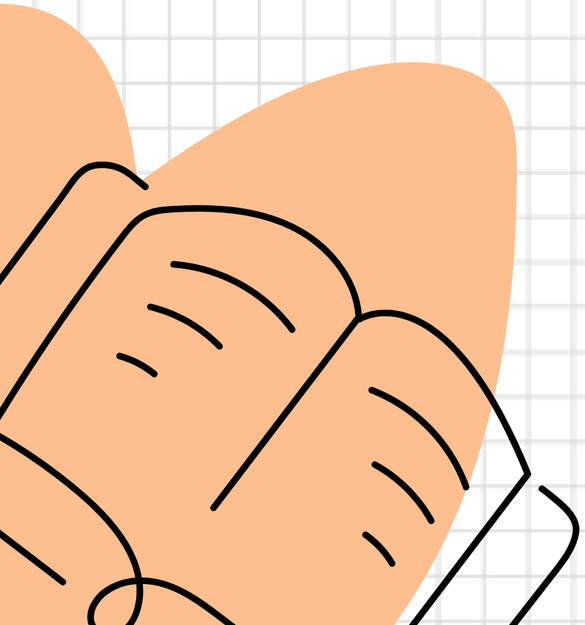
MONGOOSE METHODS



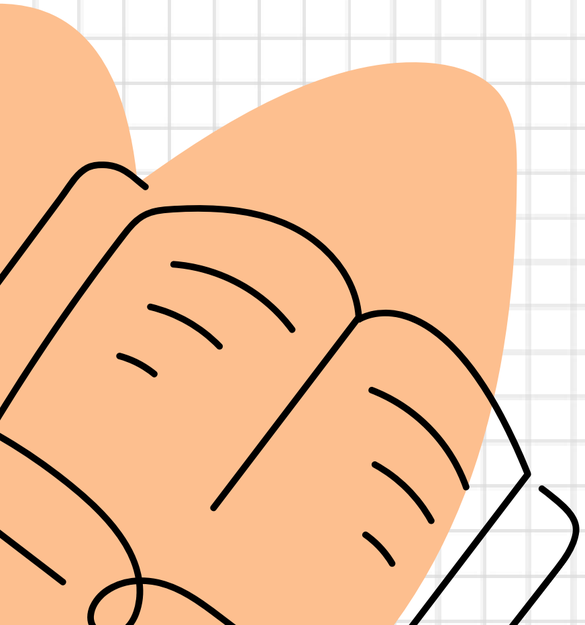
- Model.deleteMany()
- Model.deleteOne()
- Model.find()
- Model.findById()
- Model.findByIdAndDelete()
- Model.findByIdAndRemove()
- Model.findByIdAndUpdate()
- Model.findOne()
- Model.findOneAndDelete()
- Model.findOneAndReplace()
- Model.findOneAndUpdate()
- Model.replaceOne()
- Model.updateMany()
- Model.updateOne()


OH YEAH!
YOU DID IT!

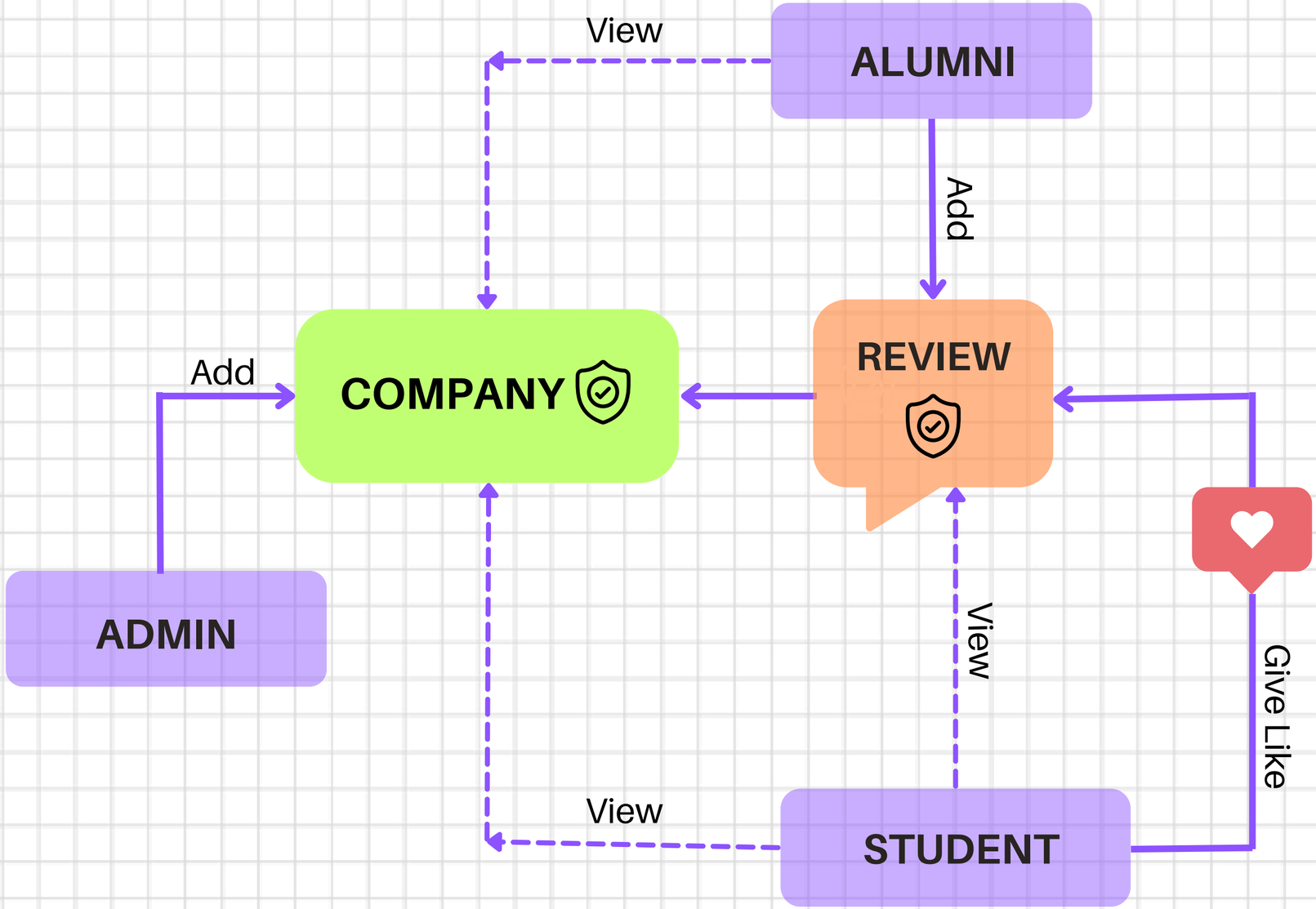
GET READY FOR MENTI QUIZ!



IT'S TIME TO START THE PROJECT



T&P COMPANY REVIEW SYSTEM



MODEL

USER

- Name
- Email
- Password
- Role

COMPANY

- Name
- Email
- Reviews [ ]

REVIEWS

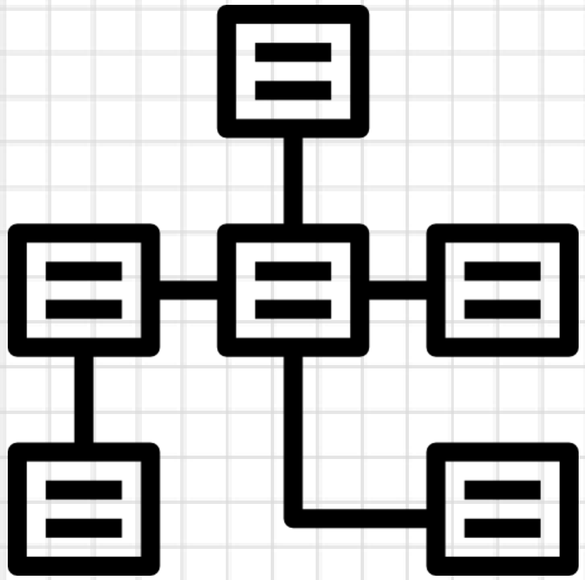
- Company
- Email
- Review
- Likes [ ]

LIKES

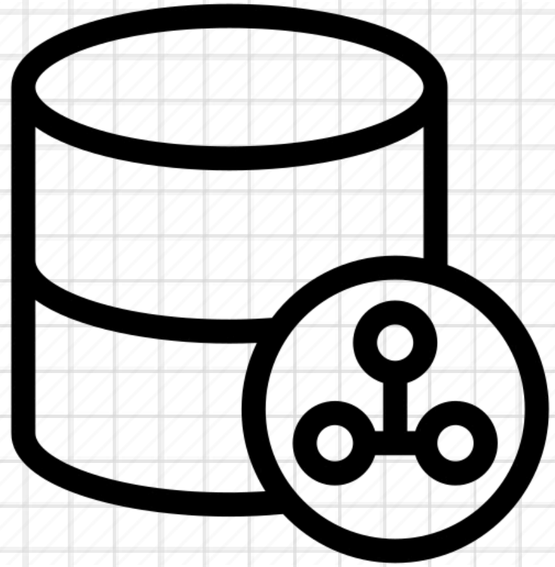
- Reviews
- Student

LET'S BUILD THE PROJECT

SCHEMA VS MODEL



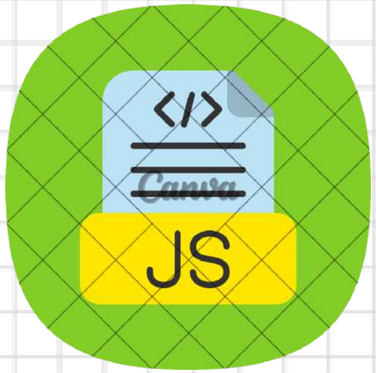
SCHEMA



MODEL

Model

Routes



index.js

View

Controller